

Python 面试题速记通关版 _ 面试刷题 mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

Python 的 range 函数如何运用? 请举例说明

`range` 是 Python 里用来生成整数序列的一个内置函数, 你压根不需要一次性把所有数放进内存, 它只是按规则“算”出下一个值, 所以特别省内存。

最常见的用法是配合 `for` 循环。比如你想打印 0 到 4 的数:

```
for i in range(5):  
    print(i)
```

这会输出 0, 1, 2, 3, 4。因为 `range(5)` 表示从 0 开始, 到 5 之前结束。

你也可以指定起始和结束位置:

```
for i in range(2, 8):  
    print(i)
```

输出 2 到 7。意思是“从 2 跑到 8 前停下”。

还能加步长。比如只取偶数:

```
for i in range(0, 10, 2):  
    print(i)
```

这就输出 0, 2, 4, 6, 8。第三个参数就是每次“跳多远”。

注意 `range` 生成的是一个可迭代对象, 不是列表。如果你真想要列表, 得手动转:

```
list(range(3)) # [0, 1, 2]
```

一般在循环计数、遍历索引时用得最多。像处理列表时想带上下标:

```
items = ['a', 'b', 'c']  
for i in range(len(items)):  
    print(i, items[i])
```

不过这种情况其实更推荐用 `enumerate`, 代码更清晰。但有些场景, 比如反向遍历索引, `range` 就很方便:

```
for i in range(len(items) - 1, -1, -1):
    print(i, items[i])
```

这就是从最后一个索引开始往前走。

什么是 Python 的位运算符？

Python 的位运算符直接对整数的二进制位进行操作，效率高，在底层计算、权限控制、数据压缩等场景用得上。

- 1) `&` 按位与：两个对应位都为 1 时结果才为 1。比如权限系统里，用一个数表示多个权限位，`perm & READ_FLAG` 就能判断是否有读权限。
- 2) `|` 按位或：任一为 1 结果就为 1。常用于权限合并，比如 `perm | WRITE_FLAG` 给用户追加写权限。
- 3) `^` 按位异或：两位不同结果为 1，相同为 0。有个经典用法是不借助临时变量交换两个数：

```
a = a ^ b
b = a ^ b
a = a ^ b
```

- 4) `~` 按位取反：每个二进制位 0 变 1、1 变 0。注意这是带符号的，`~x` 等价于 `-x-1`。
- 5) `<<` 和 `>>` 是左右移位。左移一位相当于乘 2，右移一位相当于除以 2（向下取整）。比如高性能计算中常用 `n << 3` 代替 `n * 8`。

这些运算符处理的是补码形式的二进制，所以负数要小心。像 Redis 的布隆过滤器实现就大量用了位运算来做快速判断，毕竟直接操作 bit 最省空间也最快。一般情况下不用过早优化，但了解它们能让你在看底层库代码时不吃亏。

如何在 Python 中使用多进制数字？

Python 支持直接书写和转换多进制数字，用特定前缀表示进制类型。写的时候前面加个标记就行，解释器能自动识别。

二进制以 `0b` 开头，比如 `0b1010` 就是十进制的 10。八进制用 `0o`，像 `0o12` 表示十进制 10。十六进制是 `0x`，`0xA` 还是 10。这些在位运算、权限设置或者颜色值处理时特别常见，比如 Django 或 Flask 里配置掩码时经常会看到。

如果拿到一个进制字符串想转成十进制，用 `int(str, base)` 函数。例如 `int('1010', 2)` 返回 10，`int('A', 16)` 也是 10。反过来把十进制转其他进制，用 `bin()`、`oct()`、`hex()`，返回对应前缀字符串。

注意别把八进制写成老式前导零风格，比如 `012` 在 Python 3 会报错，必须写成 `0o12` 才行。这跟 Python 2 不兼容，迁移项目时容易翻车。

```
# 示例：进制转换
print(0b1010)      # 输出 10
print(hex(255))     # 输出 '0xff'
print(int('FF', 16)) # 输出 255
```

不同进制本质都是整数，只是表示方式不同，做计算时压根不经过字符串，直接按补码运算。

Python 中如何获取字典的所有键？

Python 里想拿字典里所有的键，最直接的方式是用字典自带的 `.keys()` 方法。它返回一个 **视图对象** (dict_keys)，动态反映字典的键集合，不是列表，但能迭代。

比如有个字典：

```
user = {'name': 'Alice', 'age': 25, 'city': 'Beijing'}
keys = user.keys()
print(keys) # 输出: dict_keys(['name', 'age', 'city'])
```

这个 `keys` 是实时的，如果你后续往 `user` 里加了新键，`keys` 对象会自动更新，不用重新调用。

如果非得要列表形式，就转一下：

```
key_list = list(user.keys())
```

遍历的时候其实不需要显式调用 `.keys()`，直接 `for k in user:` 也是一样的效果，因为默认迭代的的就是键。

注意别这么写：

```
# 不推荐：手动遍历收集
keys = []
for k in user:
    keys.append(k)
```

这种写法太啰嗦，`.keys()` 就是干这活的，一行搞定。

另外，`.keys()` 在 Python 2 和 3 有区别。Python 2 返回的是列表，Python 3 是视图对象，更省内存，尤其是大字典时优势明显。现在基本都用 Python 3，这点不用担心。

为什么 Python 不建议使用下划线开头的标识符？

Python 里以下划线开头的标识符不是语法强制的访问控制，而是一种约定，用来表达属性或方法的“内部使用”意图。

1) 单下划线 `_name`

表示这个变量、函数或方法是**内部实现细节**，外部代码尽量别碰。比如模块里的辅助函数，写 `_parse_config()` 就是在说“我是工具人，请模块外的朋友无视我”。`from module import *` 会跳过它，但普通导入或直接引用还是能访问。

2) 双下划线 `__name`

这会触发名称改写 (name mangling)，在类中定义 `__x`，实际会被解释器改成 `_ClassName__x`。这是为了防止子类意外覆盖父类的私有属性。注意，它依然不是真正的私有，只是换个名字让你调用起来没那么顺手。

3) 前后双下划线 `__name__`

这是系统定义的魔术方法，比如 `__init__`、`__len__`。你可以在自定义类中实现它们来支持语言特性，但不要自己造这种名字。

其实压根没有“禁止”，但用了就等于立了 flag：改这里出问题别赖别人。像 Django 或 Flask 这些大项目，内部模块大量用单下划线区分公共 API 和内部逻辑，维护起来才不乱。

代码示例：

```
class User:
    def __init__(self, name):
        self._age = 0      # 内部状态
        self.__id = 123    # 触发 name mangling

    def _validate(self):    # 内部方法
        return True
```

真正想隐藏？压根不导出或者用闭包/描述符更靠谱。下划线只是沟通工具，不是安全机制。

Python 如何声明多个变量并赋值？

Python 里声明多个变量并赋值，其实就两种常见写法，看你是想赋相同值还是不同值。

如果多个变量要赋同一个值，比如 a、b、c 都等于 10，可以直接连等：

```
a = b = c = 10
```

这种写法三个变量指向同一个对象，在数字、字符串这种不可变类型上没问题，但要是换成列表，就得小心了，因为改一个会牵连其他变量。

更常用的是**序列解包**，一次性把多个值拆开赋给多个变量。比如：

```
x, y, z = 1, 2, 3
```

右边可以是元组、列表，只要能迭代且长度匹配就行。这种写法在交换变量时特别方便：

```
x, y = y, x
```

不用临时变量就完成了交换。

还支持扩展解包，用 `*` 接收多余元素：

```
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2,3,4], last=5
```

这种机制底层依赖的是可迭代对象的拆解，只要两边结构对得上就能赋值，写算法或者函数返回多值的时候非常顺手。

什么是 Python 元组的解封装？

Python 元组的解封装指的是把元组里的元素按顺序赋值给多个变量。写代码时很常见，比如函数返回多个值，你就可以用这个语法一口气接住。

1) 假设有个元组 `result = ('Alice', 25, 'Engineer')`，你可以直接写：

```
name, age, job = result
```

这三个变量会分别拿到元组里的对应值。这种写法简洁，也比通过索引取值更易读。

2) 解封装不局限于元组，只要是可迭代对象都能这么干，比如列表、字符串。但元组最典型，因为它是不可变的，语义上更适合用来表示一组固定字段。

3) 如果变量数量和元组长度对不上，会抛出 `ValueError`。比如左边两个变量，右边三个元素，Python 搞不定就会报错。

4) 有时候你只关心部分值，可以用下划线 `_` 占位，表示忽略某个元素：

```
name, _, job = result # 只想要 name 和 job
```

5) 还支持 `*` 操作符收集多余元素。比如：

```
first, *rest = (1, 2, 3, 4)
# first = 1, rest = [2, 3, 4]
```

这种机制在数据拆包、函数参数传递（`*args`）、交换变量值（`a, b = b, a`）里都特别实用，算是 Python 的基础操作之一。

什么是 Python? 为什么它会这么流行?

Python 是一门设计上很注重可读性的编程语言，代码写出来接近自然语言，缩进强制规范让结构清晰。这种简洁风格降低了入门门槛，初学者几天就能上手写脚本。

它流行的关键在于生态够全。科学计算有 NumPy 和 Pandas，机器学习靠 PyTorch 和 TensorFlow，Web 后端有 Django 和 Flask，自动化脚本更是随手就来。几乎每个热门领域都有成熟框架顶着，开发者不用重复造轮子。

1) 语法糖多，比如列表推导式一行搞定循环过滤

```
squares = [x**2 for x in range(10) if x % 2 == 0]
```

2) 解释型语言，跨平台运行，改完代码立即执行 3) C 扩展支持让性能敏感部分可以外挂模块，兼顾效率

社区活跃度极高，PyPI 上超 40 万包，遇到问题基本都能搜到解决方案。像数据分析团队用 Jupyter + Python 做全流程处理，从清洗到可视化一气呵成，这种协作效率在其他语言里很难复制。

企业端也买账，Instagram 用 Django 扛住千万级日活，说明它不只适合小项目。再加上教育体系普遍选用 Python 教学，每年新增大量熟悉它的开发者，形成正向循环。

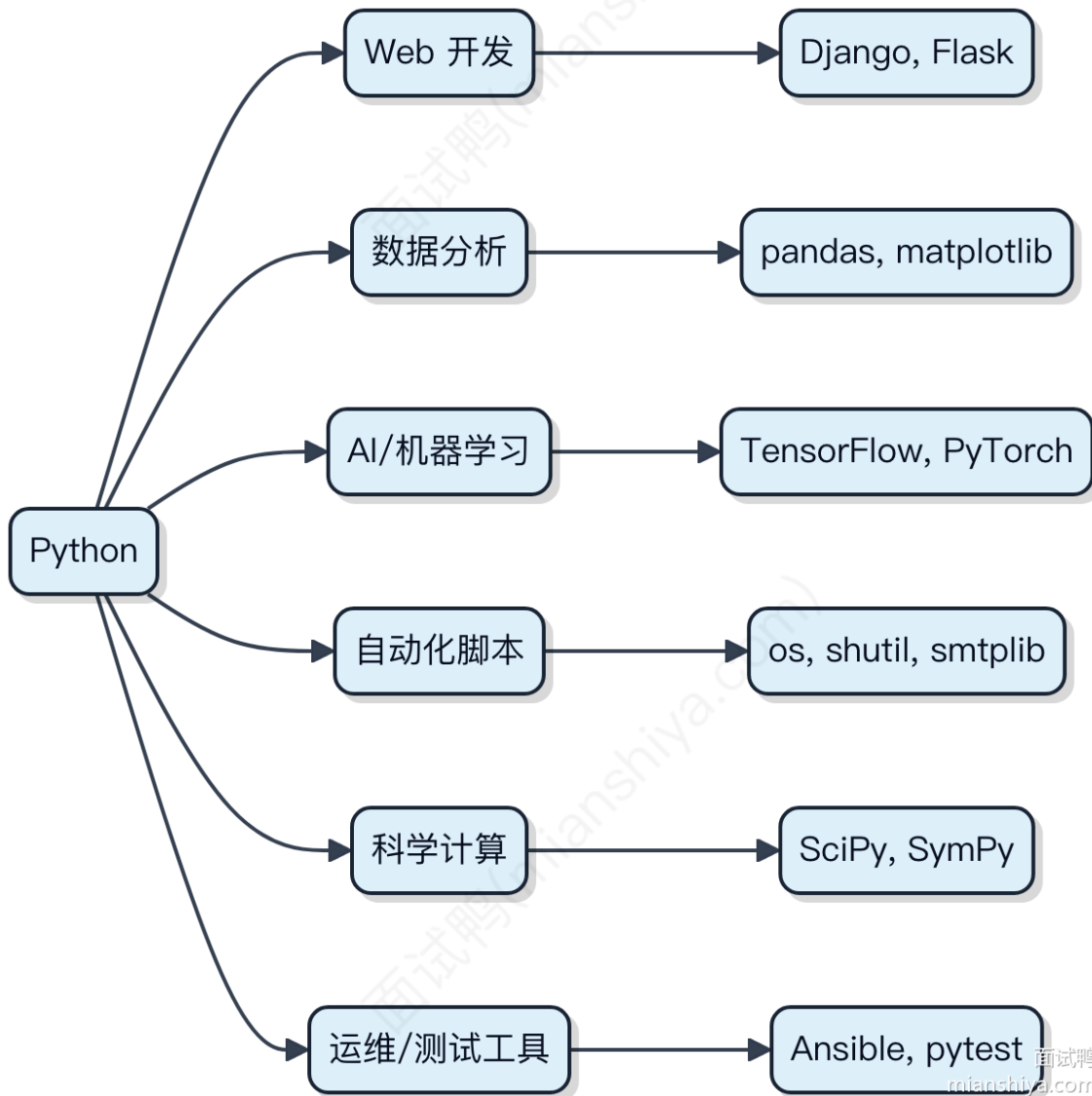
请列举一些 Python 的应用场景

Python 的应用场景其实非常广泛，主要得益于它的语法简洁和庞大的生态库。

1) Web 开发是常见方向，用 Django 或 Flask 能快速搭后台服务。比如知乎早期就是基于 Django 构建的，这类框架帮你搞定了路由、ORM 和安全机制，让你专注业务逻辑。

2) 数据处理和分析也是一大块，配合 pandas、numpy 这些库，能高效处理百万级行的数据清洗和统计。加上 matplotlib 或 seaborn，可视化图表也能直接出。

- 3) 机器学习和 AI 领域 Python 基本是首选。TensorFlow 和 PyTorch 都是 Python 优先的接口设计，研究者写模型不用纠结底层实现，几行代码就能定义神经网络。
- 4) 自动化脚本是小但高频的用法。比如批量重命名文件、定时抓日志、自动发邮件，用 os、shutil、smtplib 几个标准库就能搞定，开发成本特别低。
- 5) 科学计算和工程仿真也有身影，像 SciPy 和 SymPy 支持积分、微分方程求解，常被用于物理或数学建模场景。
- 6) 测试和运维工具链里也常见，比如 Ansible 就是纯 Python 写的配置管理工具，通过 SSH 控制上千台服务器，脚本容易写也容易维护。



Python 有哪些局限性?

Python 的短板主要集中在性能和并发模型上，这跟它的设计取舍直接相关。

1) 执行效率问题

Python 是解释型语言，代码逐行解释执行，没有编译成机器码。像数值计算、频繁循环这种场景，比 C/C++ 慢 10 到

100 倍很常见。虽然有 Cython 或 Numba 可以局部加速，但核心瓶颈在解释器本身。

2) GIL (全局解释器锁) 限制并发

GIL 是 CPython 解释器的机制，它让同一时刻只有一个线程执行 Python 字节码。这意味着多线程在 CPU 密集型任务中根本发挥不了多核优势。比如用 threading 模块跑计算密集任务，你会发现 CPU 只有一个核在跑满。想真正并行得靠 multiprocessing，但进程间通信成本高，内存不共享。

3) 移动和嵌入式支持弱

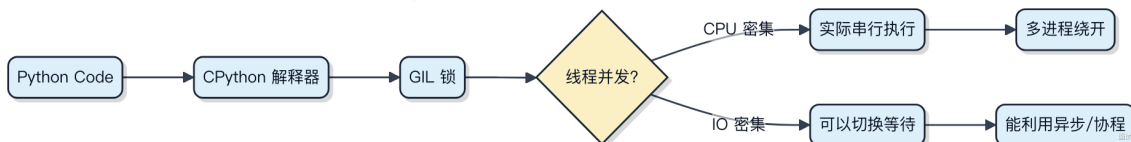
不像 Java 有 Android SDK，Python 在移动端基本没戏。Kivy 这类框架尝试过，但生态和性能都撑不起来。嵌入式设备资源有限，CPython 解释器本体就十几 MB，压根不适合塞进 MCU。

4) 打包和分发麻烦

pyinstaller 打包出来的应用动辄几十 MB，启动还慢。不同系统依赖的动态库容易出问题，尤其是涉及 numpy、pandas 这种带 native extension 的包。

5) 类型系统松散，大型项目维护难

虽然 3.5+ 引入了 type hint，但属于运行时可选检查。不像 Go 或 Rust 那样强制编译期类型安全，团队做大项目时容易写出隐蔽 bug，重构也提心吊胆。



所以 Web 后端用 Django/Flask 没问题，但高并发网关或实时数据处理，一般会换 Go 或 Java 来扛住。

请解释 Python 代码的执行过程？

Python 代码执行不是直接跑在 CPU 上的，它走的是“编译成字节码 + 虚拟机解释执行”这套流程。你写完 .py 文件后，Python 解释器会先把它编译成 .pyc 字节码，存在 __pycache__ 里。

这些字节码是 CPython 虚拟机认识的指令，接下来由 **CPython 虚拟机** 一条条解释执行。整个过程是单线程的，靠 **GIL (全局解释器锁)** 保证同一时刻只有一个线程执行字节码，所以多线程在 CPU 密集任务里压根不经过并行提速。

模块第一次导入时才会触发编译，之后加载 .pyc 就快了。比如你用 Django 或 Flask，启动时会看到 __pycache__ 目录生成一堆文件，就是这个阶段留下的。

代码示例，看看字节码长啥样：

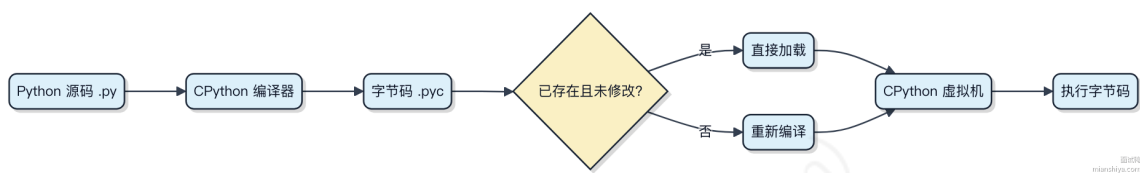
```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

输出会展示 LOAD_FAST、BINARY_ADD 这些虚拟机指令，跟汇编有点像。

如果你用的是 PyPy，那走的就是 JIT 编译路线，热点函数会被编译成本地机器码，执行效率更高。而 CPython 主打一个稳定和兼容性，脏活累活都扛住了，但性能上限摆在这。



Python 有哪些内置数据结构?

Python 的内置数据结构主要就这几种，日常写代码基本都绕不开它们。

列表（list）用来存一组有序的数据，能改，也能重复。比如你从数据库查出一堆用户记录，用 list 装最合适。

```
users = ['Alice', 'Bob', 'Charlie']
users.append('David')
```

元组（tuple）跟 list 很像，但不能改。适合当字典的 key 或者固定配置项。

```
coord = (3, 5)
config = ('localhost', 8080)
```

字典（dict）是键值对，底层是哈希表实现的，查找快得飞起，平均 O(1)。处理 JSON、缓存映射、统计计数这些场景全靠它。

```
user_info = {'name': 'Alice', 'age': 30}
```

集合（set）存不重复的元素，也是基于哈希表，常用去重或者做成员判断。比如你要筛掉重复的 IP 访问记录：

```
unique_ips = set(['1.1.1.1', '2.2.2.2', '1.1.1.1'])
```

字符串（str）也算一种不可变序列，支持切片操作，虽然功能多，但别拿它拼接大量文本，效率很差，用 `join()` 才是正道。

这几种结构里，**可变性和有序性**是关键区别点。list 和 dict 可变，tuple 和 frozenset 不可变；list、tuple、str 有序，dict 在 Python 3.7+ 才正式保证插入顺序。

- 1) 可变：list、dict、set
- 2) 不可变：tuple、str、frozenset
- 3) 无序：set、dict（3.7 前）
- 4) 有序：list、tuple、str、dict（3.7+）

Python 中单引号和双引号有什么区别?

Python 里单引号和双引号在字符串定义上完全等价，选哪个纯看个人或团队风格。

你写 'hello' 还是 "hello" 解释器压根不关心，效果一模一样。这一点跟 Java 或 C 不同，Python 并不区分字符和字符串，也没有语法层面的差异。

但实际用的时候，大家会约定一些习惯。比如 JSON 字符串里本身有双引号，外面就用单引号包，省得转义：


```
data = '{"name": "小明", "age": 18}'  
# 可以写成  
data = '{"name": "小明", "age": 18}' # 外层单引号
```

反过来，如果字符串里单引号多，比如英文句子带缩写，外层用双引号更省事：

```
text = "I can't believe it"
```

另外 f-string 必须用双引号开头，除非你里面不包含双引号。其实也不是必须，只是混合使用能少写转义符：

```
name = "Alice"  
f'Hello, {name}' # OK  
f"Hello, {name}" # 也 OK  
f"He said \"hi\""  
f'He said "hi"' # 所以外层单引号更清爽
```

总结一下，语法没区别，**灵活性**才是关键。项目里统一一种风格就行，别来回切换，代码看起来才干净。

Python 中 break、continue、pass 有什么作用？

break 是用来直接跳出当前循环的。比如你在 for 或 while 循环里，条件满足了想立刻停止，就用 break，后面剩下的迭代压根不执行了。

continue 是跳过当前这一次循环的剩余语句，直接进入下一轮循环。适合在某些条件下不想处理当前数据，但又不希望终止整个循环的情况。

pass 是个空操作语句，当语法上需要一条语句但实际不需要做任何事的时候就用它占位。比如写函数骨架时还没实现细节，先 pass 顶着，不然 Python 会报缩进错误。

代码示例：

```
for i in range(5):  
    if i == 2:  
        continue  
    if i == 4:  
        break  
    print(i)
```

输出是 0, 1, 3。因为 i=2 时跳过了 print，i=4 时循环直接结束。要是把 continue 换成 pass，输出就是 0, 1, 2, 3 —— pass 不影响流程，该打印还是打印。

这几个关键字通常出现在条件判断里，控制循环行为。别把 break 和 return 搞混了，return 是退出函数，break 只退循环。在嵌套循环中，break 只跳出最内层，不会一层层全退。

Python 中如何实现 switch 语句？

Python 官方直到 3.10 版本才通过 **match-case** 语法原生支持类似 switch 的结构。在这之前，开发者得靠其他方式模拟。

3.10+ 的写法很直观：

```
def handle_status(code):
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not Found"
        case 500 | 503:
            return "Server Error"
        case _:
            return "Unknown"
```

这个 match-case 不只是值匹配，还能解构数据。比如处理元组或对象时特别方便：

```
match point:
    case (0, 0):
        print("原点")
    case (x, y) if x == y:
        print(f"在 y=x 线上，位置是 {x}, {y}")
    case (_, _):
        print("二维点")
```

老版本 Python 常用字典映射函数来模拟：

```
def switch_case(status):
    return {
        'start': start_func,
        'pause': pause_func,
        'stop': stop_func
    }.get(status, default_func())
```

这种方式干净，但所有函数都会被提前求值，不适合带参数或需要延迟执行的场景。更复杂的可以结合 getattr 做动态分发，像 Flask 的路由调度就有这味道。

总的来说，简单枚举用字典，复杂模式匹配用 match-case。老项目兼容考虑还是得靠 if-elif 链，不过代码一长就容易看花眼。

Python3 中 is 和 == 有什么区别？

is 和 == 看起来都是比较，但它们干的事完全不是一回事。

== 比的是值，也就是两个对象的内容是不是一样。比如两个字符串 "hello"，哪怕它们是从不同地方创建的，内容一样，== 就返回 True。

而 is 比的是身份，也就是这两个变量是不是指向内存中的同一个对象。它判断的是 id(a) == id(b)，也就是底层的对象地址是否一致。

举个例子：

```
a = [1, 2, 3]
b = [1, 2, 3]
```

```
print(a == b) # True, 内容相同
print(a is b) # False, 是两个不同的列表对象
```

但有些情况下，Python 会对小整数或短字符串做缓存，这时候可能会“碰巧”是同一个对象：

```
a = 256
b = 256
print(a is b) # True, 小整数在 [-5, 256] 被缓存

a = 257
b = 257
print(a is b) # 可能是 False, 不一定被缓存（取决于实现和上下文）
```

所以一般情况下，判断值相等用 `==`，只有当你明确想确认两个变量是否指向同一个对象时才用 `is`。比如判断是不是 `None`，就该用 `if x is None`，因为 `None` 是单例，这种写法更安全也更快。

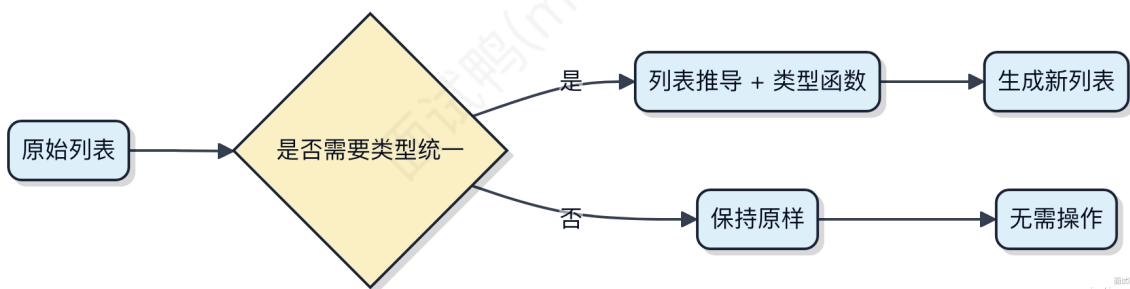
常见误区是拿 `is` 去比字符串或数字，结果在某些环境对、某些环境错，这就是因为依赖了 Python 的对象缓存机制，这不靠谱。

如何更改 Python 列表的数据类型？

Python 列表本身不要求元素类型一致，所以列表里可以同时存在整数、字符串、布尔值等不同类型的数据。但如果你希望把列表中所有元素统一转成某种特定类型，比如把字符串列表转成整数列表，通常有几种做法。

- 1) 用列表推导式配合类型转换函数，这是最常见也最推荐的方式。比如原列表是 `['1', '2', '3']`，想转成整数，可以直接写 `[int(x) for x in lst]`。这种方式简洁高效，控制力强。
- 2) 如果数据量特别大，可以考虑用 NumPy 的 `astype()` 方法。先把列表转成 NumPy 数组，然后调用 `astype(int)` 或 `astype(str)` 来批量转换类型。适合科学计算场景，性能比纯 Python 更好。
- 3) 使用 `map()` 函数也能实现类似效果，比如 `list(map(int, lst))`。不过可读性略差，尤其嵌套时不如列表推导直观。

需要注意的是，原始列表不会被修改，这些操作都会生成新列表。另外，如果原数据中有无法转换的项，比如把 `'abc'` 转 `int`，程序会直接抛出 `ValueError`。处理前最好做校验或用 `try-except` 包裹。



Python 中怎么注释代码？

Python 里的注释方式其实就三种，用对了能让代码自己说话。

- 1) 单行注释用 `#`，后面跟说明文字，这是最常用的。比如：

```
# 计算用户年龄
age = 2023 - birth_year
```

2) 多行注释其实没有专门语法，一般用连续的 `#` 行，或者用三引号字符串。注意，三引号 `'''` 或 `"""` 本意是定义多行字符串，不是语言级别的注释，但常被拿来当块注释用，尤其是在函数开头写文档时：

```
"""
这是一个处理订单的函数
参数: order_id 用户订单编号
返回: 处理结果布尔值
"""

def process_order(order_id):
    return True
```

这种三引号写法其实就是 **docstring**，会被 `help()` 和文档工具识别，所以别乱写。

3) 注释不是越多越好。变量名能表达意图的，比如 `total_price`，就不用写“这是总价”。真正需要注释的是“为什么这么做”，比如：

```
# 支付超时设为15秒，因为第三方接口最长响应时间是10秒
timeout = 15
```

另外，别把废弃代码用注释“注掉”，版本管理工具才是干这事儿的。注释只用来解释逻辑意图。

Python 是否有 main 函数？

Python 没有强制的 `main` 函数，不像 Java 或 C++ 那样要求程序入口必须是 `main` 方法。脚本从上到下直接执行，哪段代码在顶层就先运行哪段。

但大家常看到 `if __name__ == "__main__":`：这种写法，其实这就是一种约定俗成的 `main` 入口。当 Python 文件被直接运行时，`__name__` 就会被设为 `"__main__"`；如果是被导入（import），就不会触发这块逻辑。

这个模式主要是为了区分“作为脚本运行”和“作为模块被引用”的场景。比如你写了个工具函数，既想单独运行测试，又想让别人能 `import` 到别的文件里用，这时候就把测试代码塞进 `if __name__ == "__main__":` 里。

```
def process():
    print("处理数据")

if __name__ == "__main__":
    process()
```

这样别人 `import` 你的文件时，`process()` 不会自动执行，只有你自己直接运行这个文件才会走 `if` 里的逻辑。

所以结论是：Python 没有语法层面的 **main 函数**，但通过 `__name__` 机制实现了等效的控制能力，属于语言设计上的灵活性体现。

Python 中如何使用索引反转字符串？

Python 里反转字符串最常用的方式是切片，直接用 `[::-1]` 就能搞定。这个操作底层由 C 实现，效率很高，日常用起来也特别顺手。

比如你有一个字符串 `s = "hello"`，写成 `s[::-1]`，结果就是 `"olleh"`。这里的三个参数分别是起始、结束和步长，省略前两个表示全范围，-1 作为步长代表从后往前取。

当然也有其他办法，比如用 `reversed()` 配合 `join()`：

```
s = "hello"
reversed_s = ''.join(reversed(s))
```

这种方式语义清晰，但性能略低一点，因为涉及函数调用和迭代器封装，在短字符串上差别不大，长了就明显了。

还见过有人用递归或者 for 循环拼接，那种写法在实际开发中基本不会用，一是可读性差，二是字符串不可变的特性决定了每次拼接都会创建新对象，n 次操作就是 $O(n^2)$ 时间和内存开销，搞不定大文本。

所以一般情况下，切片是最推荐的做法，简洁又快。只有在需要逐字符处理的同时反转时，才考虑用 `reversed()` 这种带迭代器的方式。

为什么 Python 中没有函数重载？

Python 没有函数重载，根本原因在于它的动态类型机制和命名空间的设计方式。

函数重载要求同一个作用域内能存在同名但参数不同的函数，调用时根据参数类型或数量决定具体执行哪一个。Java 这类静态语言在编译期就能确定调用哪个方法签名。但 Python 是运行时绑定，函数名本质是一个变量名，指向一个调用对象。

你定义一个同名函数，其实就是把原来的函数对象引用覆盖掉了。比如：

```
def add(a, b):
    return a + b

def add(a, b, c):
    return a + b + c
```

第二个 `add` 一定义，前面那个就找不到了。压根不经过“重载解析”这一步，直接就是后者替换了前者。

不过 Python 提供了替代方案。你可以用默认参数处理可选参数的情况：

```
def add(a, b, c=None):
    return a + b if c is None else a + b + c
```

更复杂的场景可以用 `functools.singledispatch` 实现基于参数类型的分发，或者用 `*args`，`**kwargs` 动态处理参数。像 FastAPI 这种框架就靠类型注解和运行时检查实现类似重载的效果，但底层还是单个函数在处理。

说白了，Python 的设计哲学是“一种明显的方式”，不像 C++ 或 Java 那样依赖编译期的多态机制。

你知道哪些 Python 魔术方法？

Python 的魔术方法本质上是类的特殊方法，以双下划线开头和结尾，用于拦截并自定义对象的行为。它们不是魔法，只是 Python 解释器预留的钩子。

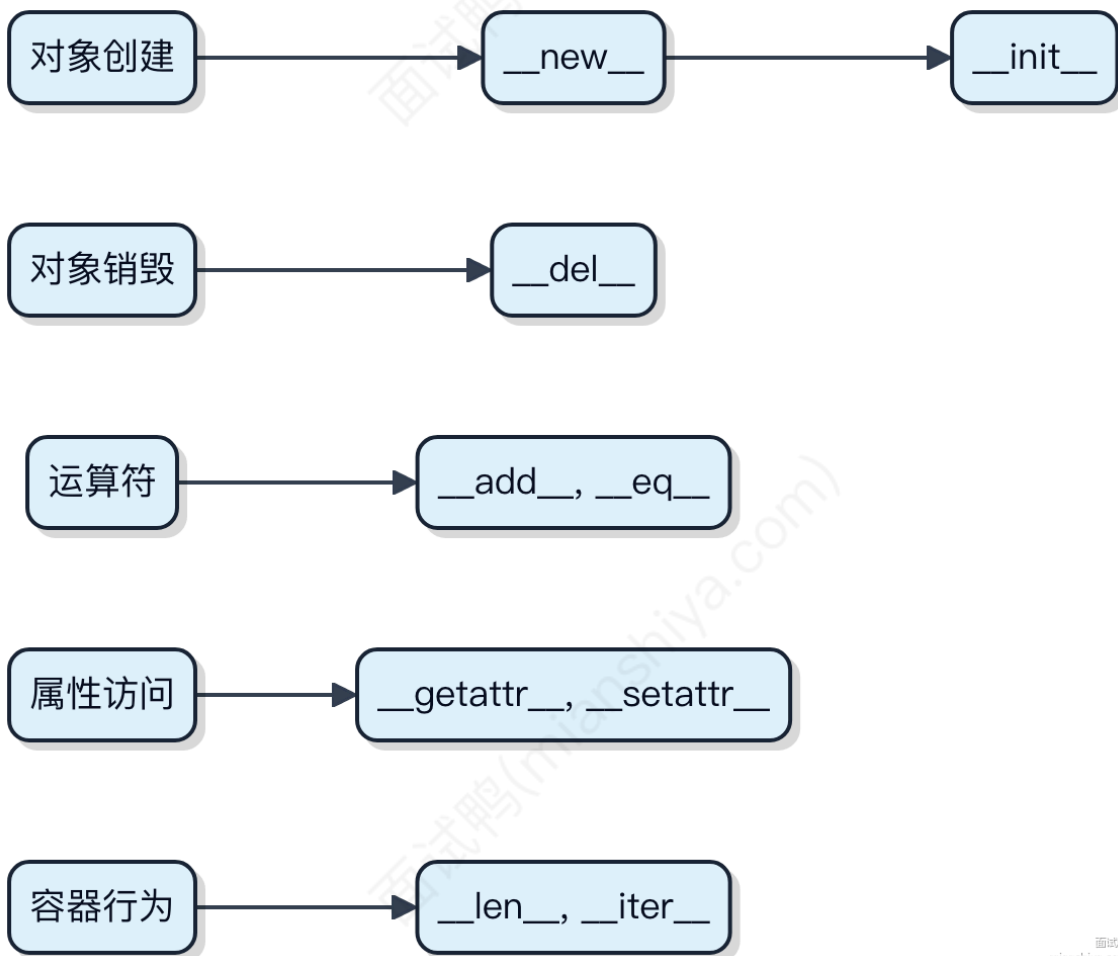
比如创建实例时，`__new__` 先被调用，它负责返回一个实例，而 `__init__` 负责初始化数据。这两个方法配合完成“构造”逻辑。如果想控制实例创建过程，比如实现单例，可以在 `__new__` 里动手脚。

对象销毁前会触发 `__del__`，但别依赖它做资源清理，因为无法保证何时触发。更推荐上下文管理器配合 `__enter__` 和 `__exit__`，像文件操作那样能确保释放资源。

运算符重载也靠魔术方法。`__add__` 对应 `+`，`__eq__` 控制相等判断，`__lt__` 决定排序行为。有了这些，自定义类就能像内置类型一样自然使用操作符。

访问属性时，`__getattr__`、`__getattribute__`、`__setattr__` 可以介入逻辑。比如实现懒加载或日志追踪。但 `__getattribute__` 会拦截所有访问，一不小心就死循环，得小心用。

容器行为由 `__len__`、`__getitem__`、`__iter__` 等定义。实现了它们，你的类就能用 `len()`、支持下标访问、可迭代。Django 的 `QuerySet` 就是靠这套机制实现惰性求值。



面试鸭
mianshiya.com

请介绍 Python 中变量的作用域？

Python 里的变量作用域决定了在哪个位置能访问到某个变量。这个规则其实就靠一套叫 LEGB 的查找顺序来控制。

- 1) L (Local) 指的是当前函数内部。你在一个函数里用 `x = 10` 定义的变量，就只在这个函数里生效。
- 2) E (Enclosing) 是外层函数的作用域。比如嵌套函数，内层函数可以读取外层函数定义的变量。

3) G (Global) 是全局作用域，也就是模块文件顶层定义的变量，任何函数都能读它。

4) B (Built-in) 是内置作用域，像 `print`、`len` 这些不用导入就能用的名称。

查找时从近到远：先看局部有没有，再往外层函数找，接着是全局，最后查内置。但赋值行为会改变这一逻辑 —— 只要函数内部对变量用了 `=` 赋值，默认就被当成局部变量，除非你用 `global` 或 `nonlocal` 明确声明。

举个例子：

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        nonlocal x
        x = "modified"
    inner()
```

这里 `nonlocal` 告诉解释器别创建新的局部变量，而是去外层找 `x`。要是没这句，`x = "modified"` 就会在 `inner` 里新建一个局部变量，外层压根不改。

如何在 Python 中实现字符串替换操作？

字符串替换在 Python 里最常用的就是 `replace()` 方法，直接生成新字符串，原字符串不变。传两个参数，老子串换新子串，还能加第三个参数控制替换次数。

1) 基础用法像这样：

```
text = "hello world"
new_text = text.replace("world", "Python")
# 结果是 "hello Python"
```

2) 想只替换前几个匹配项，比如最多换 2 次：

```
text = "a-b-c-d"
new_text = text.replace("-", "_", 2)
# 结果是 "a_b_c-d"，只有前两个被替换
```

别忘了还有 `str.translate()` + `str.maketrans()` 组合，适合批量替换多个字符，性能更好。比如把所有标点换成下划线：

```
text = "hello, world! how are you?"
translator = str.maketrans(",!?", "_")
text.translate(translator)
# 结果是 "hello_ world_ how are you_"
```

正则也行，`re.sub()` 更灵活，能处理模式匹配。比如统一替换各种空白符：

```
import re
text = "hello \t world\n"
```

```
re.sub(r"\s+", " ", text).strip()
# 多个空格、制表符、换行全压成一个空格
```

一般情况下 `replace()` 足够，简单直接。要批量换字符优先考虑 `translate`，规则复杂再上 `re.sub`。这三招覆盖了绝大多数场景。

你知道哪些 Python 的编码规范？

Python 的编码规范主要看 PEP 8，这是官方认可的代码风格指南。写 Python 代码时，大家默认都往这个标准靠。

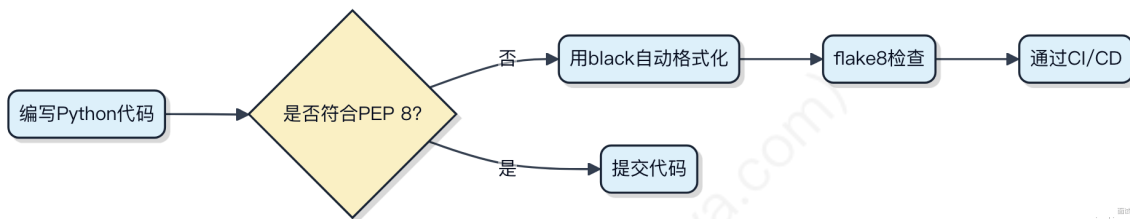
命名上，函数和变量用小写字母加下划线，比如 `get_user_info`；类名用大驼峰，像 `UserProfile`；常量全大写加下划线，例如 `MAX_TIMEOUT = 30`。

缩进统一用 4 个空格，别用 `tab`。行长度控制在 79 字符以内，方便并排看代码。导入语句每行一个，按标准库、第三方库、本地模块分组，组间空一行。

布尔值判断直接用真假上下文，别写 `if flag == True:`，应该写 `if flag:`。字符串拼接多的时候用 `f-string`，比如 `f"Hello, {name}"`，比 `%` 或 `.format()` 更直观。

有个工具叫 `black`，能自动格式化代码，基本不用手动调格式。`flake8` 可以检查是否符合 PEP 8 规范，CI 流程里加上这两，团队协作不容易出岔子。

- 1) 函数参数不要用可变对象做默认值，比如 `def func(items=[])`，这个列表会跨调用共享，容易出 bug
- 2) 注释要解释“为什么”，而不是“做什么”，代码自己该说明后者
- 3) 模块层级的变量、函数写完就跑，别堆逻辑到 `if __name__ == "__main__":` 里



init 方法在 Python 中有什么作用？

`__init__` 方法是 Python 中用来初始化新创建对象的方法。每次通过类创建实例时，它会自动调用，把属性设置好。

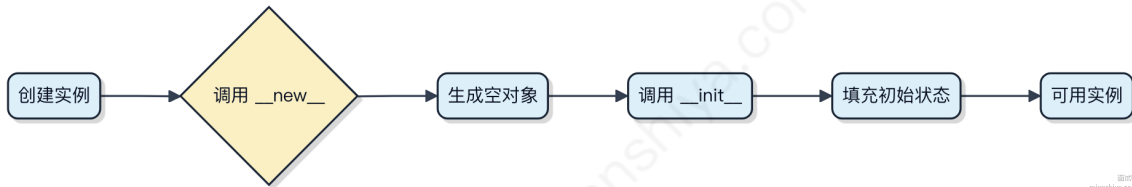
- 1) 它不负责创建对象，真正的构造过程由 `__new__` 完成，但绝大多数情况下我们不用碰 `__new__`，所以 `__init__` 就是实际使用中配置对象状态的入口。
- 2) 你可以把它看作是类的“启动脚本”，比如给实例变量赋初值。像定义一个用户类时，在 `__init__` 里把名字、年龄这些传进来并绑定到 `self` 上。

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

- 3) 参数不是必须的，但建议根据业务需要设计签名。如果不需要传参，也别忘了写 `self`。

4) 它返回 `None` 。如果你在里面 `return` 了别的东西, Python 会报错。这一点和很多语言的构造函数不一样, 得注意。

5) 继承时要注意调用父类的 `__init__` , 否则基类的初始化逻辑会被跳过。常见写法是用 `super().__init__()` 显式触发。



什么是鸭子类型 (duck typing) ?

鸭子类型不是说真有只鸭子, 而是一种动态语言里判断对象能力的方式。它不关心对象是什么类型, 只看它能不能“嘎嘎叫”——也就是有没有对应的方法或属性。

比如一个函数需要调用对象的 `quack()` 方法, 只要传进来的对象有这个方法, 哪怕它是企鹅类, 也能通过。这在 Python、Ruby 这类语言里很常见。

1) Python 示例:

```
def make_it_quack(duck):
    duck.quack()

class Duck:
    def quack(self):
        print("Quack!")

class Dog:
    def quack(self):
        print("Woof? But I'm pretending to be a duck.")

make_it_quack(Duck()) # 正常输出 Quack!
make_it_quack(Dog()) # 也能跑, 输出 Woof? But I'm pretending...
```

这种设计让代码更灵活, 写通用函数特别方便, 比如序列化、遍历接口。但代价是 IDE 难以静态分析, 运行时才暴露问题。

Java 是静态类型, 编译时就得确定类型关系, 所以压根不经过鸭子类型的路。你要实现某个接口, 必须明明白白地 `implements` 。像 Spring 的自动装配、MyBatis 的 Mapper 接口, 都是靠声明契约来工作的。

对比来看, 鸭子类型适合快速迭代的小项目或脚本, 而大型系统更依赖显式契约来维持可维护性。你得想清楚, 到底是追求灵活性, 还是可控性。

你使用过哪些 Python 标准库模块?

Python 标准库其实挺庞大的, 日常开发里真正高频使用的模块就那么几十个。挑几个关键的说说。

处理文件和路径，`os` 和 `pathlib` 基本就够用了。`pathlib` 更现代，写起来也清爽，比如 `Path('/tmp').joinpath('a.txt')` 比字符串拼接靠谱多了。

数据结构方面，`collections` 模块里的 `defaultdict`、`Counter`、`deque` 经常用。比如统计词频直接 `Counter(words)` 一行搞定，比手写字典方便太多。

网络请求一般用第三方库 `requests`，但偶尔轻量场景也会用 `urllib.request`，毕竟不用装包。`json` 模块更是天天见，序列化反序列化基本都靠它。

时间处理绕不开 `datetime`，解析时间字符串或者计算时间差都很顺手。需要定时任务时会用到 `time.sleep()` 配合循环，复杂点的调度就得上 `schedule` 这类第三方库了。

调试和日志首选 `logging`，比 `print` 强太多了，能分级能输出文件。单元测试自然离不开 `unittest`，虽然很多人转用 `pytest`，但标准库这个够基础。

- 1) `re` 处理正则表达式，文本清洗时少不了
- 2) `random` 生成随机数，写 demo 或抽样都用得上
- 3) `argparse` 写命令行工具必备，参数解析很直观

这些模块加起来覆盖了文件、数据、网络、时间、日志等常见需求，基本能满足大部分脚本和工具开发。标准库的优势是开箱即用，部署不依赖外部环境。

说明 Python 中的 zip 函数

Python 的 `zip` 函数其实是个“打包工”，把多个可迭代对象按位置一一对应组合成元组。你给它几个列表，它就返回一个迭代器，每次取一个元组，包含每个列表的第 `i` 个元素。

比如你有两个列表 `names = ['Alice', 'Bob']` 和 `ages = [25, 30]`，`zip(names, ages)` 拿到的就是 `('Alice', 25)`，`('Bob', 30)` 这样的序列。常用来构造字典或者并行遍历。

```
dict(zip(['a', 'b'], [1, 2])) # {'a': 1, 'b': 2}
```

它不生成新列表，返回的是迭代器，省内存。一旦用过一轮就空了，第二次遍历会得到空结果，这点容易踩坑。

如果传进去的可迭代对象长度不一样，`zip` 默认以最短的为准。想处理不等长的情况，可以用 `itertools.zip_longest`，短的自动填默认值。

```
list(zip([1, 2], ['a'])) # [(1, 'a')] — 第二个被截断
```

反过来解包也常用，比如把一组坐标对拆开：

```
pairs = [(1, 'a'), (2, 'b')]
nums, letters = zip(*pairs) # nums=(1,2), letters=('a','b')
```

这招在数据预处理时特别顺手，比如用 `Pandas` 或 `NumPy` 前做结构转换。

如何使用 Python 的 random 模块生成随机数、实现随机乱序和随机抽样？

Python 的 `random` 模块是处理随机性任务的主力工具，日常写脚本、做模拟、搞数据抽样都绕不开它。

生成随机数最常用的是 `random.random()`，返回 0 到 1 之间的浮点数。如果要指定范围，直接上 `random.uniform(a, b)`。需要整数？用 `random.randint(a, b)`，闭区间，两边都包含。

想打乱列表顺序，`random.shuffle()` 就地操作，直接修改原列表。比如洗牌场景，用它再合适不过。注意它是副作用函数，不会返回新列表。

随机抽样靠 `random.sample(population, k)`，从总体中无放回抽取 k 个不同元素。比如从 100 个用户里随机选 5 个发奖，一行搞定。要是允许重复抽，就改用 `[random.choice(population) for _ in range(k)]`。

```
import random

nums = [1, 2, 3, 4, 5]
random.shuffle(nums) # nums 被打乱

sampled = random.sample(range(1, 101), 5) # 从 1-100 抽 5 个
picked = random.choice(['apple', 'banana', 'orange']) # 随机选一个
```

需要注意，这些生成器都是伪随机，基于种子确定序列。调试时可以固定 `random.seed(42)` 让结果可复现。生产环境一般不用管，系统会自动初始化种子。

对于高安全需求，比如生成令牌或密钥，别用 `random`，得换 `secrets` 模块，它才是真·随机熵源。

什么是 Python 的成员运算符？

Python 的成员运算符用来判断一个元素是否存在于某个容器类型中，主要有两个：`in` 和 `not in`。它们返回布尔值，常用于字符串、列表、元组、集合和字典这类可迭代对象。

比如你想知道某个值在不在列表里，不用写循环遍历比对，直接用 `in` 就行。

```
fruits = ['apple', 'banana', 'orange']
print('banana' in fruits) # True
print('grape' not in fruits) # True
```

查字典时，默认是判断键（key）是否存在，而不是值。

```
d = {'name': 'Alice', 'age': 25}
print('name' in d) # True
print(25 in d) # False, 因为 25 是值，不是键
```

对于字符串，`in` 判断的是子串是否包含。

```
print('he' in 'hello') # True
```

这玩意儿底层其实是调用对象的 `__contains__` 方法。你自定义的类如果想支持这个操作，实现这个方法就行。

```
class MyContainer:
    def __init__(self, data):
        self.data = data

    def __contains__(self, item):
```

```
        return item in self.data

c = MyContainer([1, 2, 3])
print(2 in c) # True
```

一般情况下，集合（set）和字典的查找效率是 $O(1)$ ，用 `in` 很快；而列表、元组这种是线性扫描，数据一多就慢，超过几千项就得小心性能问题。

Python 中 any() 和 all() 方法有什么作用？

`any` 和 `all` 是 Python 里用来判断可迭代对象中元素真假的内置函数，用起来简单但很实用。

`any(iterable)` 只要 `iterable` 里**有一个**元素为真，就返回 `True`。比如你查一堆任务有没有完成的，有个完成的就行。

```
any([False, False, True]) # 返回 True
```

`all(iterable)` 要求 `iterable` 里**所有**元素都为真，才返回 `True`。典型场景是权限校验，得全部通过才行。

```
all([True, True, False]) # 返回 False
```

空列表的情况容易忽略：

- `any([])` 返回 `False`，因为“一个为真的都没有”
- `all([])` 返回 `True`，这叫“空真”，逻辑上说得通——“没有不为真的元素”

常见用法像是过滤数据时做条件判断：

```
# 检查是否存在负数
has_negative = any(x < 0 for x in numbers)

# 检查是否全是正数
all_positive = all(x > 0 for x in numbers)
```

这两个函数不会遍历整个序列到底，一旦结果确定就短路返回。比如 `any` 遇到第一个 `True` 就停，`all` 遇到第一个 `False` 就停，性能上比较友好。

说明 Python3 中装饰器的用法

装饰器本质是个函数，用来在不修改原函数代码的前提下，给它加功能。比如你有个函数要测执行时间，或者要加日志，用装饰器就特别顺手。

写法上，装饰器接收一个函数作为参数，返回一个包装后的函数。Python 用 `@` 语法糖来应用装饰器，写在函数定义前头。

```
def timer(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} 耗时: {time.time() - start:.2f}s")
    return wrapper
```

```

        return result
    return wrapper

@timer
def slow_func():
    time.sleep(1)

slow_func() # 输出耗时信息

```

带参数的装饰器稍微绕一点，其实是三层函数。第一层接收装饰器参数，第二层接收原函数，第三层执行逻辑。

```

def retry(times=3):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for i in range(times):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    if i == times - 1:
                        raise e
            return wrapper
        return decorator

@retry(times=2)
def call_api():
    # 模拟不稳定的接口
    import random
    if random.random() < 0.8:
        raise ConnectionError()
    print("API 调用成功")

```

实际项目里，Flask 的路由、Django 的权限控制、functools.lru_cache 都是装饰器的典型应用。写中间件或 SDK 时也常用来统一处理异常、日志这些脏活累活。

Python 有哪些特点和优点？

Python 的设计哲学是“优雅、明确、简单”，它的特点和优点主要体现在开发效率和生态上。

语法简洁直观，强制缩进让代码结构清晰，省去了大量括号和分号。写同样功能的逻辑，Python 通常比 Java 或 C++ 少 30% 到 50% 的代码量，适合快速原型开发。

动态类型系统让变量无需声明类型，用起来灵活，但大型项目里容易出错，需要靠测试和类型注解（如 typing 模块）来补足。

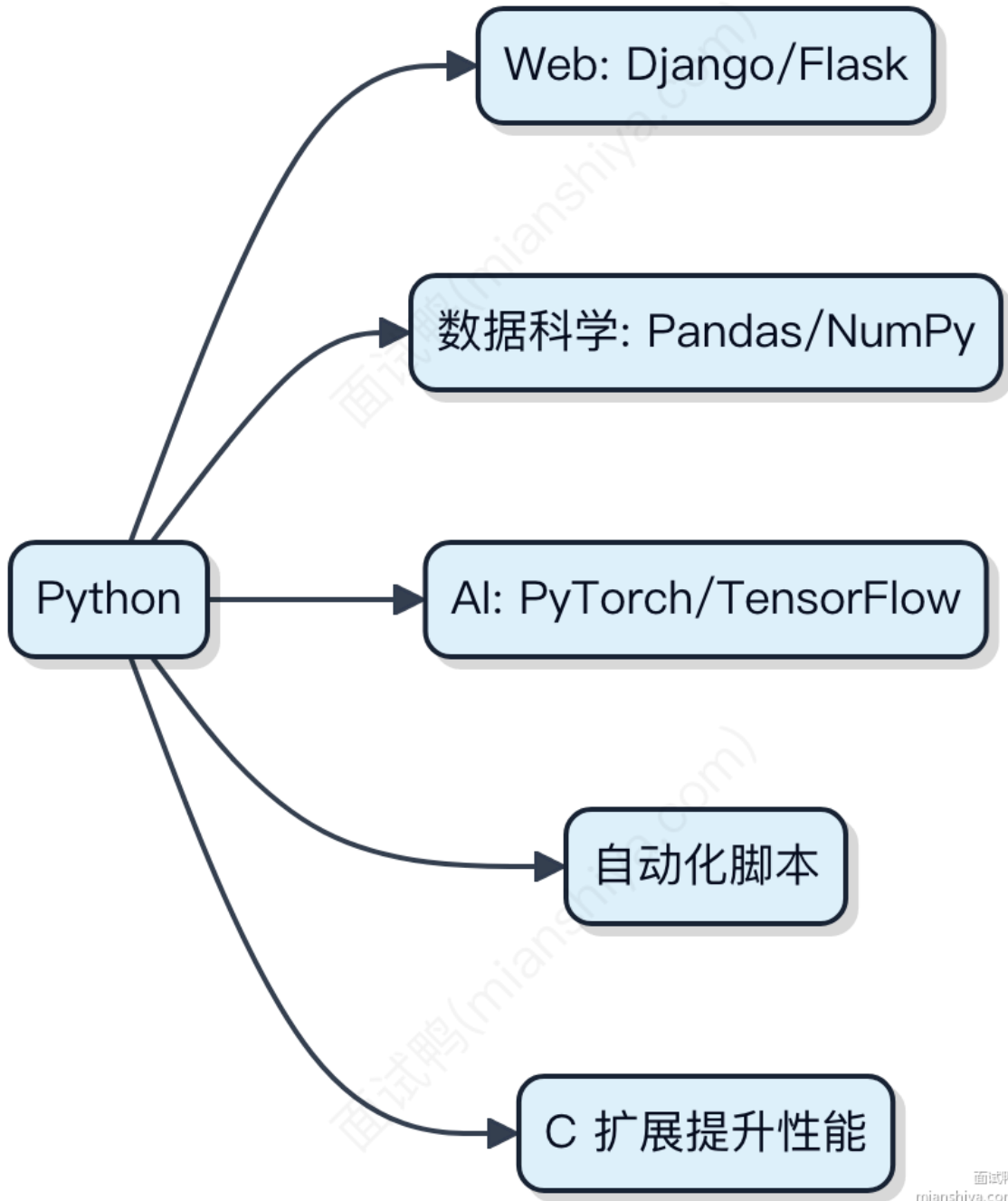
它有极强的**标准库**，比如 os、json、http 这些开箱即用，做 Web 用 Flask 或 Django，搞数据分析上 Pandas 和 NumPy，机器学习直接上 PyTorch 或 TensorFlow，基本主流场景都有轮子。

解释执行，跨平台，但性能不如编译型语言。对 CPU 密集任务，可以用 Cython 或集成 C 扩展来加速。GIL 存在导致多线程不能真正并行，CPU 密集场景得靠多进程扛住。

- 1) 胶水语言，能轻松调用 C/C++ 库
- 2) 社区活跃，第三方包丰富，pip 安装几乎覆盖所有常见需求

3) 广泛用于自动化脚本、数据处理、AI、Web 后端、爬虫等

不适合硬实时或高并发长连接服务，比如网关或消息中间件，这种场景 Go 或 Java 更稳。



总的来说，Python 把开发者的生产力放在第一位，牺牲一点运行效率换来的是上线速度和维护成本的优势。

解释 Python 中 `//`、`%` 和 `**` 运算符

`//` 是向下取整的除法，比如 `7 // 3` 结果是 `2`，而 `-7 // 3` 是 `-3`，注意它不是简单地截断小数，而是向负无穷方向取整。这点和很多语言不同，容易出错。

`%` 是取模运算，结果的符号只取决于**右操作数**。例如 `-7 % 3` 得到 `2`，因为 Python 的 `%` 和 `//` 是配套设计的，满足 $(a // b) * b + (a \% b) == a$ 这个恒等式。这个特性在做循环索引、哈希分片时特别有用，比如用 `%` 实现环形缓冲区下标自动回绕。

`**` 是幂运算，`2 ** 10` 就是 `1024`，支持小数次幂和负数次幂，比如 `9 ** 0.5` 算平方根，结果是 `3.0`。它优先级高于加减乘除，但低于括号。

这三个运算符在数学计算、算法题里经常一起出现。比如快速幂可以用递归配合 `**` 和 `//` 实现：

```
def pow_mod(x, n, mod):
    if n == 0:
        return 1
    half = pow_mod(x, n // 2, mod)
    return (half * half * (x if n % 2 else 1)) % mod
```

注意 `n // 2` 和 `n % 2` 配合来做奇偶判断和拆分，这是常见套路。浮点数用 `**` 要小心精度问题，整数运算则基本不会翻车。

什么是 Python 中的三元表达式？

Python 中的三元表达式是一种简洁的条件赋值语法，用来在一行内根据条件决定变量的取值。它不是独立的语句，而是一个表达式，能嵌入到其他代码结构中。

写法是：`value_if_true if condition else value_if_false`。程序会先判断 `condition`，成立就返回前面的值，否则取后面的。

比如写一个判断用户是否成年的例子：

```
age = 20
status = "adult" if age >= 18 else "minor"
```

这比写完整的 if-else 块紧凑多了，特别适合初始化变量或构造简单逻辑。但别嵌套太多层，像 `A if C1 else B if C2 else D` 这种读起来就费劲了，超过两层最好拆成函数或普通条件语句。

它常用于列表推导式里做条件筛选：

```
numbers = [1, 2, 3, 4]
tags = ["even" if x % 2 == 0 else "odd" for x in numbers]
```

这种场景下用三元表达式既清晰又高效。不过记住，它的核心优势是**表达力**，不是性能，可读性优先。

请简单介绍 Python 的 Flask 框架，有什么作用？

Flask 是一个用 Python 写的轻量级 Web 框架，适合快速搭建小型服务或 API。它不强制项目结构，也没有自带 ORM 或认证模块，你要啥可以自己配，比如用 SQLAlchemy 做数据库操作，用 Werkzeug 处理请求响应。

1) 核心是路由和视图函数，你定义一个 URL 路径，绑定一个函数处理逻辑，返回字符串或 JSON。2) 底层基于 Werkzeug 实现 WSGI 应用，请求进来后由它封装成 Request 对象，响应通过 Response 返回。3) 支持模板渲染，用 Jinja2 引擎，适合做简单的服务端页面生成。

```

from flask import Flask, jsonify

app = Flask(__name__)

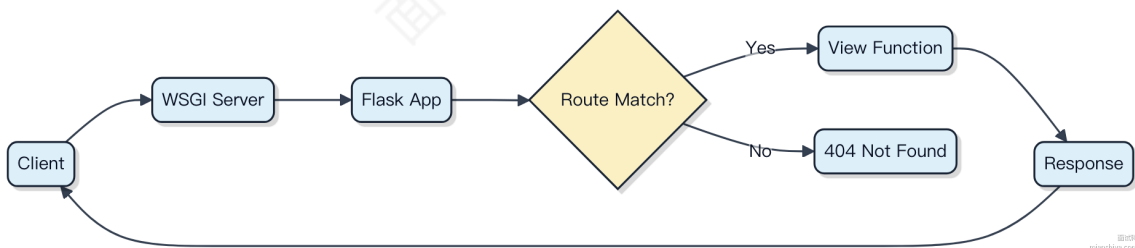
@app.route('/hello')
def hello():
    return jsonify(message="Hello from Flask!")

# 启动服务: app.run()

```

这种灵活性让它在原型开发、微服务、教学场景里很常见。比如写个内部工具接口，几行代码就能跑起来。但项目一大就容易混乱，得靠开发者自己组织代码结构。

对比 Django 这种“大而全”的框架，Flask 更像是搭积木，基础块给你，其他你自己选。要加配置管理、蓝图拆分、中间件，都得手动来，好处是没冗余功能，轻装上阵。



如何在 Python 中管理内存？

Python 的内存管理对开发者来说大部分是自动化的，但理解背后机制能帮你写出更高效、少 bug 的代码。

Python 用引用计数作为主要的内存回收手段。每个对象都有个计数器，记录有多少变量在引用它。一旦引用数归零，比如变量被删除或超出作用域，内存立刻被释放。这很高效，能快速回收临时对象。

不过引用计数搞不定循环引用的问题。比如两个对象互相引用，即使没其他变量用它们，引用数也不为零。这时候就得靠垃圾回收器（GC），它会定期扫描并清理这类不可达对象。你可以用 gc 模块手动触发或调整策略，但在 CPython 里一般不需要。

对于大量小对象的场景，比如 Web 服务中频繁创建 dict 或 list，Python 还用了对象池和缓存机制。像小整数（-5 到 256）、短字符串会被驻留，重复使用同一内存地址，减少分配开销。

想控制内存行为？可以借助 __slots__ 减少实例属性的字典开销，或者用生成器替代大列表，避免一次性加载全部数据到内存。

```

class Point:
    __slots__ = ['x', 'y'] # 节省内存，避免每个实例都建 __dict__

```

总的来说，日常开发不用太操心内存，但处理大数据、长时间运行服务时，得留意循环引用和对象生命周期，必要时用 sys.getsizeof() 或 tracemalloc 分析内存使用。

Python 中 help() 函数和 dir() 函数有什么作用？

`help()` 和 `dir()` 是 Python 中两个非常实用的内置函数，用来快速了解对象的信息，尤其在调试和学习阶段特别有用。

`dir()` 主要是列出一个对象所有可用的属性和方法。比如你导入了一个陌生模块，又不知道它有什么功能，直接 `dir(module_name)` 就能看到里面都有哪些类、函数和变量。对类或实例调用 `dir()` 时，它会返回这个对象的 `__dict__` 和所属类的属性，帮你快速摸清结构。

```
dir(list)
# 输出: ['__add__', '__class__', '__contains__', ..., 'append', 'clear', 'copy', ...]
```

`help()` 更进一步，它不只是列名字，而是给你详细的说明文档。它会调出对象的 docstring，告诉你这个函数怎么用、参数是什么、返回什么。相当于在 REPL 里直接查官方文档。

```
help(str.split)
# 输出: split(sep=None, maxsplit=-1) -> list
#       Return a list of the words in the string...
```

一般来说，先用 `dir()` 看“有哪些”，再用 `help()` 看“怎么用”。这两个函数在交互式环境里是探索未知代码的标配组合，压根不需要翻网页就能搞清楚一个对象的基本用法。

Python 程序退出时，是否释放所有内存分配？

Python 程序退出时，操作系统会回收进程占用的全部内存，这是系统层面的保障。我们讨论的重点其实是 Python 解释器自身是否主动释放所有对象。

对于 CPython 来说，大部分对象靠引用计数管理。只要引用计数归零，内存立刻被释放。但存在例外：循环引用的对象，即使不再使用，引用计数也不为零，无法被及时清理。

这类对象依赖 **垃圾回收器**（GC）来处理。CPython 的 GC 是分代回收机制，程序退出前通常会触发一次完整的垃圾回收，尝试清理这些不可达对象。不过这个过程不保证 100% 执行，尤其在异常终止或调用 `os._exit()` 时可能跳过。

另外，有些内存压根不经过 Python 的内存池，比如通过 C 扩展直接申请的内存，或者解释器内部结构占用的空间。这部分能否释放取决于对应模块的实现。像 NumPy 数组、用 ctypes 分配的缓冲区，正常退出时一般会被清理，但也不能完全依赖 Python 主动释放。

代码里可以手动触发清理：

```
import gc
gc.collect() # 强制运行垃圾回收
```

但实际开发中，没必要在退出前写一堆 `del` 或 `gc.collect()`。操作系统会兜底回收资源，把精力放在解决内存泄漏和优化运行期的对象生命周期更实在。

简单说，能释放的 Python 对象基本都会被处理，但别指望它像堆栈弹出那样精确。最终的内存安全是由操作系统托底的。

说明 Python3 中 yield 的用法

`yield` 关键字出现在函数里时，这个函数就变成了生成器。调用它不会立即执行，而是返回一个迭代器对象，每次调用 `next()` 才会运行到下一个 `yield` 位置。

1) 普通函数 return 一次就完了, 而带 yield 的函数可以暂停和恢复。比如做大数据量的读取, 你不想一次性 load 到内存, 就可以一行行 yield 出去。

```
def read_large_file(f):  
    for line in f:  
        yield line.strip()
```

2) yield 能保持局部状态。下次从 yield 后继续, 变量值都还在。这比写类维护状态简洁多了。

3) 可以用在协程中, 通过 send() 给生成器传值。这时候 yield 既能产出值也能接收值, 实现双向通信。

```
def accumulator():  
    total = 0  
    while True:  
        value = yield total  
        if value is not None:  
            total += value
```

这种模式被 asyncio 用来模拟异步任务调度, 虽然现在更推荐用 async/await。

4) 注意不能跟 return 带值混用 (语法允许但逻辑混乱), 而且 list(yield_from_func()) 会把所有值吐出来, 大集合小心撑爆内存。

其实 generator 解决的是资源和控制流的问题。数据不需要全加载、流程需要暂停恢复时, yield 就派上用场了。像 Flask 的请求上下文、Scrapy 爬虫 pipeline 都能看到类似思想。

解释型语言 Python 和编译型语言有什么区别?

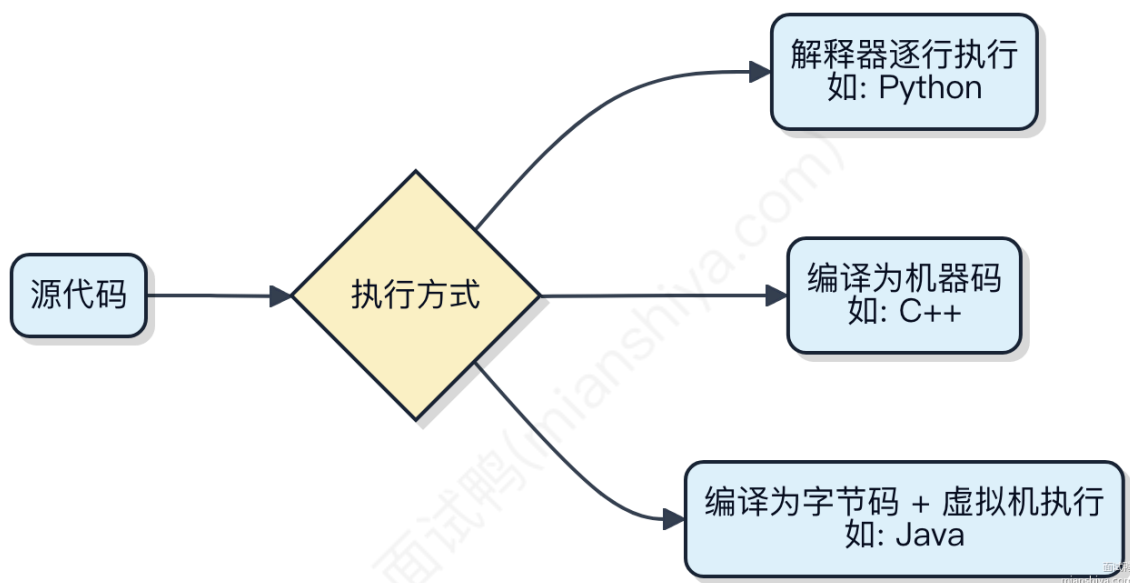
解释型语言和编译型语言最根本的区别在于代码执行的方式, Python 是典型的解释型语言, 而像 Java、C++ 这类属于编译型。

Python 代码运行时, 解释器会逐行读取源码, 边翻译边执行, 整个过程不生成独立的二进制可执行文件。你写完 .py 文件, 直接 python script.py 就能跑, 开发调试特别快。但代价是执行效率相对低, 因为每次运行都要重新解析。

像 C++ 这种传统编译型语言, 得先用编译器把整个源码编译成机器码, 产出一个可执行文件, 比如 Linux 下的二进制文件。之后运行就不再需要源码或编译器, 直接执行, 速度快, 资源占用少。但跨平台麻烦, Windows 编的不能直接扔到 Linux 跑。

Java 其实是个中间路线, 它先编译成字节码, 再由 JVM 解释或即时编译 (JIT) 执行。所以既有编译过程, 又有解释执行的特性。

1) 解释型适合快速迭代、脚本任务, 比如用 Python 写自动化工具、数据分析。2) 编译型适合性能敏感场景, 比如高频交易系统、游戏引擎、操作系统组件。



Python 的灵活性强，但搞不定高并发或低延迟的系统级任务；C++ 反过来，性能猛，开发成本也高。选哪种，看你要的是开发效率还是运行效率。

说明 Python 中 enumerate() 的用法

遍历容器时，如果既要元素又要索引，别自己用 `range(len())` 拼了，`enumerate` 就是干这个的。它返回一个迭代器，每次产生一个 (index, value) 的元组。

1) 基本用法直接套在可迭代对象上就行

```
for i, item in enumerate(['a', 'b', 'c']):
    print(i, item)
# 输出: 0 a, 1 b, 2 c
```

2) 起始索引可以自定义，不用非从 0 开始

```
for i, item in enumerate(['a', 'b', 'c'], start=1):
    print(i, item)
# 输出: 1 a, 2 b, 3 c
```

这玩意儿在列表推导式里也挺常见，比如想找出所有偶数位置的元素

```
data = ['x', 'y', 'z', 'w']
result = [v for i, v in enumerate(data) if i % 2 == 0]
# 得到 ['x', 'z']
```

其实底层就是个语法糖加优化过的 C 实现，性能比手动维护计数器好一点，关键是代码更干净。遇到需要“第几个”这种语义的时候，优先考虑 `enumerate`，别再写 `i = 0; while ...` 那一套了。

什么是 Python 的负索引?

Python 的负索引是从序列尾部开始倒着数的取值方式。比如最后一个元素用 -1，倒数第二个是 -2，以此类推。

你写 `my_list[-1]`，拿到的就是列表最后一个元素。这在处理动态长度的数据时特别方便，不用先算 `len(my_list) - 1`。

- 1) -1 指向最后一个
- 2) -2 指向倒数第二个
- 3) 一直往前推，直到 `-len(seq)`，再小就会越界报错

切片也支持负索引。`my_list[-3:]` 表示从倒数第三个到末尾，`my_list[:-2]` 是从开头到倒数第二个之前的元素。

常见场景是取文件路径的最后一级，比如 `path.split('/')[-1]` 拿文件名，或者日志处理时取最近几条记录。

```
arr = [10, 20, 30, 40]
print(arr[-1]) # 输出 40
print(arr[-2]) # 输出 30
```

超过范围会抛 `IndexError`。像 `arr[-5]` 对于一个只有 4 个元素的列表就搞不定了。

这种设计让代码更简洁，尤其是做数据清洗或字符串处理时，省去了大量长度计算。

Python 是否区分大小写？

Python 是区分大小写的，从变量名、函数名到类名，大小写不同就代表不同的标识符。

比如定义一个变量 `name = "Alice"`，再用 `Name` 或 `NAME` 去访问，解释器会报 `NameError`，因为它认为这是三个完全不同的名字。函数也一样：

```
def say_hello():
    print("Hello")

# 调用 say_Hello() 会报错，压根找不到这个函数
```

类名遵循同样的规则。定义 `class MyClass:` 后，写成 `myclass` 就没法实例化它。模块导入也是如此，`import MyModule` 和 `import mymodule` 可能指向不同文件，取决于操作系统和路径实际存在的情况。

这规则贯穿整个语言设计，写代码时必须严格匹配大小写，不然搞不定。

不仅标识符，关键字虽然都是小写，但你要是写个 `If` 或 `For`，也会因为找不到对应语法而报错。所以写 Python 时，别指望它自动忽略大小写差异，该咋写就咋写。

说明 Python 中标识符的命名规则

Python 的标识符命名看着简单，其实有几条硬性规定得遵守。

标识符就是变量名、函数名、类名这些你自己起名字的地方。首先，只能用字母、数字和下划线，而且**第一个字符不能是数字**。比如 `name`、`user_age` 都行，但 `2name` 就不行。中文字符其实也可以作为开头，比如 `姓名 = "张三"`，语法上是合法的，但一般不推荐，团队协作容易出问题。

Python 对大小写敏感，`myName` 和 `myname` 是两个不同的标识符。另外，不能用关键字当标识符，比如 `if`、`for`、`class` 这些，编辑器通常会高亮提示。

还有一个约定俗成的规则：以下划线开头的名字有特殊含义。单下划线 `_name` 表示内部使用，不是 public API；双下划线 `__name` 会触发名称改写（name mangling），主要用于类的私有属性；前后都双下划线的像 `__init__` 是系统定义的特殊方法，别自己乱造这种名字。

总的来说，合法只是第一步，写代码还得考虑可读性。一般用小写字母加下划线的组合，比如 `user_list`、`get_config`，类名则用大驼峰 `UserProfile`，这是社区普遍接受的风格。

Python 中如何删除字符串中的前置空格？

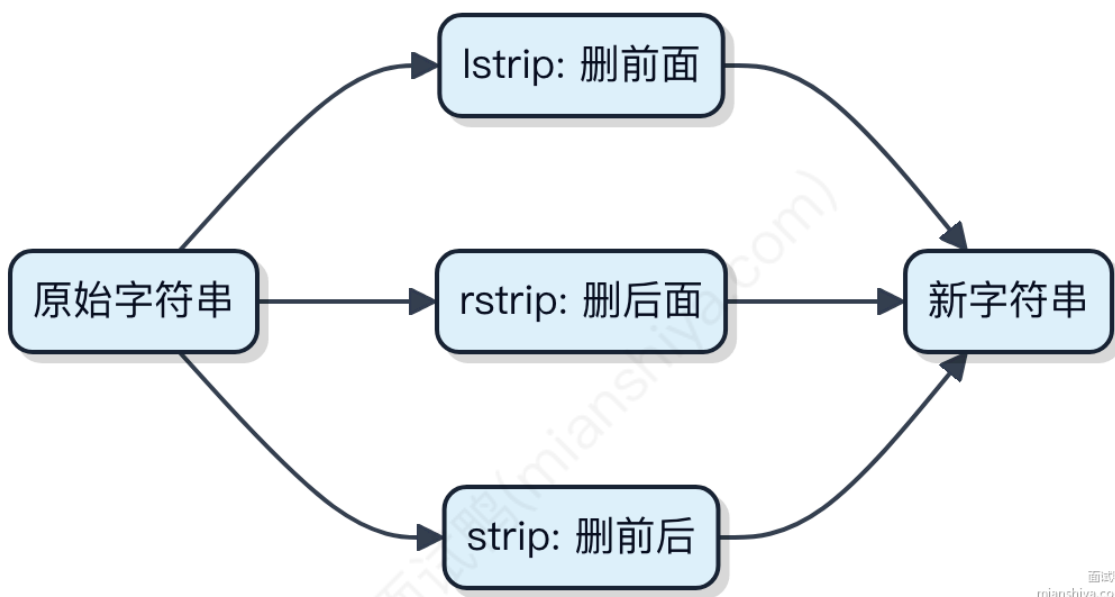
Python 字符串自带的 `lstrip()` 方法就是专门处理这个的，调用后会把字符串开头的所有空白字符都去掉。

- 1) 直接调用 `lstrip()`，不带参数，会移除所有前置空格和制表符、换行符这类空白字符
- 2) 也可以传个字符集，比如 `lstrip(' ')` 就只删空格，不会动别的空白符

```
text = "  hello world  "
clean = text.lstrip() # 结果是 "hello world  "
```

注意它返回的是新字符串，原字符串不会被修改，因为字符串在 Python 里是不可变的。你要用变量接住返回值才算真正“删除”。

如果想同时删前后空格，用 `strip()`，删后置空格用 `rstrip()`。这三个方法配合使用基本能覆盖所有场景。



别自己写循环去处理，这种基础操作标准库已经优化得很熟了，直接用最稳。

Python 中如何将字符串转换为小写？

Python 里把字符串转小写，直接调用字符串对象的 `lower()` 方法就行。这个方法会返回一个新字符串，原字符串不变。

比如你有个变量：

```
text = "Hello World"
result = text.lower()
print(result) # 输出: hello world
```

`lower()` 对 ASCII 字符没问题，像 A-Z 这种都能正确转成 a-z。它 also 支持 Unicode 字符，比如带重音符号的字母，在多数情况下也能按规则转成对应的小写形式。

需要注意的是，如果字符串里压根没有大写字母，调用 `lower()` 也不会报错，就直接返回原内容的副本。

有些场景下你还得考虑地域化问题，比如土耳其语里的 "I" 转小写不是简单的 "i"，这时候可能需要更复杂的处理逻辑，但绝大多数情况用不到。

另外别忘了，字符串是不可变类型，所以任何转换都不会改变原始变量，必须接住返回值用新的变量来接。

- 1) 调用 `lower()` 得到新字符串
- 2) 原字符串不受影响
- 3) 适合常规英文文本处理，比如搜索、去重、统一格式等场景

如果你只是想判断两个字符串在忽略大小写时是否相等，可以直接用：

```
if text1.lower() == text2.lower():
    print("不区分大小写时相等")
```

Python 的 pass 语句有什么作用？

`pass` 是个空操作语句，写上去就跟没写一样，解释器遇到它直接跳过，不干任何事。但这个“啥也不干”恰恰是它的价值所在。

你写代码时，有些地方语法上要求必须有语句，但逻辑上又不想处理，比如定义函数或类的时候，还没想好具体实现。这时候你不能空着，Python 会报缩进错误。写个 `pass` 就能占个位置，让结构合法。

- 1) 用在未完成的函数里

```
def todo_function():
    pass
```

- 2) 用在条件分支的空处理

```
if flag:
    print("已处理")
else:
    pass # 暂时不需要处理
```

- 3) 用在类的占位

```
class MyStubClass:
    pass
```

其实它就像交通里的“此路不通但路基留着”，程序跑过去不会出错，后续可以继续扩展。没有它的话，你得写个 `print("")` 或者 `None` 来凑数，反而显得不专业。所以 `pass` 的本质是**语法占位符**，解决的是“必须写点什么”的

问题，而不是“要做点什么”。

什么是 Python 的身份运算符？

Python 的身份运算符用于判断两个变量是否指向同一个对象，也就是它们在内存中的位置是否相同。这和比较值是否相等是两回事。

- 1) `is` 和 `is not` 就是身份运算符。`a is b` 返回 `True` 仅当 `a` 和 `b` 实际上是同一个对象，`is not` 是它的否定。
- 2) 它底层比较的是对象的 `id`，也就是 `id(a) == id(b)`。你可以把 `id` 理解成对象在内存中的地址编号。
- 3) 别把它和 `==` 混了。`==` 比较的是值，而 `is` 比较的是“是不是同一个东西”。比如两个列表 `[1,2] == [1,2]` 是 `True`，但 `[1,2] is [1,2]` 通常是 `False`，因为它们是两个不同的对象实例。

代码上看：

```
a = [1, 2]
b = a
c = [1, 2]

print(a is b) # True, b 是 a 的引用
print(a is c) # False, c 是另一个对象
print(a == c) # True, 值一样
```

一般你用 `is` 最多的场景是判断是否为 `None`，比如 `if obj is None:`。这是标准写法，因为 `None` 是单例，不会有两个 `None` 对象。

小陷阱：小整数和短字符串可能会被 Python 缓存（驻留），所以 `a = 1000; b = 1000; a is b` 可能是 `False`，但 `a = 1; b = 1; a is b` 会是 `True`。这种依赖实现细节的行为别依赖。

什么是 Python 的关系运算符？

Python 的关系运算符用来比较两个值的大小或是否相等，结果是布尔类型 `True` 或 `False`。这些运算符不会改变原始数据，只做判断。

常见的关系运算符有：

- 1) `==` 判断两边值是否相等。比如 `5 == 5` 返回 `True`，而 `"hello" == "world"` 是 `False`。注意不要和赋值符号 `=` 搞混。
- 2) `!=` 判断两边值是否不相等。例如 `3 != 4` 为 `True`，而 `5 != 5` 为 `False`。
- 3) `<` 和 `>` 分别表示小于和大于。像 `10 > 3` 成立，返回 `True`；`2 < 1` 不成立，返回 `False`。
- 4) `<=` 和 `>=` 是小于等于和大于等于。比如 `x <= 100` 在 `x` 为 99 或 100 时都返回 `True`。

这些运算符常用于条件控制中，比如 `if` 语句或循环判断。你可以把它们看作数学里的比较符号，只不过在代码里写法更明确。

关系运算的结果经常作为程序跳转的依据。比如根据用户输入的分数的判断是否及格：

```
score = 85
if score >= 60:
    print("及格")
```

所有关系运算符的优先级低于算术运算符，所以像 `a + 1 > b - 2` 这种表达式会先算加减，再做比较。

什么是 Python 的赋值和算术运算符？

Python 的赋值和算术运算符是写代码最基础的操作，天天都用。

算术运算符就是加减乘除这些。 `+` 做加法， `-` 做减法， `*` 是乘法， `/` 是浮点除法，比如 `7 / 2` 得 `3.5`。要是想整除，用 `//`，`7 // 2` 结果是 `3`。取余用 `%`，`7 % 3` 是 `1`。幂运算用 `**`，`2 ** 3` 就是 `8`。

这些运算符不光能用于数字，还能用在其他类型上。比如字符串可以用 `+` 拼接，`'a' + 'b'` 得 `'ab'`；列表也能用 `+` 合并，`[1] + [2]` 变成 `[1, 2]`。

赋值运算符最基础的是 `=`，把右边的值给左边变量。`x = 5` 就是把 `5` 赋给 `x`。还有复合赋值，比如 `+=`、`-=`、`*=`，它们能简化写法。`x += 3` 等价于 `x = x + 3`，既读又写一次变量。这类操作在循环计数或者累加时特别常见。

```
count = 0
count += 1 # 相当于 count = count + 1
```

注意一点，Python 没有 `++` 或 `--` 自增自减操作符，想加一就得写 `+= 1`。

总的来说，算术运算处理计算逻辑，赋值运算保存结果，两者配合支撑起程序的基本运行。

什么是 Python 的逻辑运算符？

Python 的逻辑运算符用来组合或反转布尔表达式的值，主要是 `and`、`or`、`not` 这三个。

1) `and` 要求两边都为真，结果才为真。比如 `x > 0 and x < 10`，只有 `x` 在 `0` 到 `10` 之间时才返回 `True`。一旦左边为 `False`，右边压根不执行，这叫短路求值。

2) `or` 是只要有一个为真就行。像 `x < 0 or x > 10`，`x` 只要超出范围就成立。如果左边已经为 `True`，右边就不跑了。

3) `not` 是取反，`not True` 就是 `False`。它优先级最低，一般建议用括号明确意图，比如 `not (x == y)`。

这些运算符不一定要返回布尔值，而是返回实际操作的值。比如 `'a' or 'b'` 返回 `'a'`，因为空字符串以外的值都算 `True`，所以 `and` 和 `or` 会直接返回操作数本身。

这特性常被用来做默认值赋值，例如 `name = input_name or 'default'`，如果 `input_name` 为空就用 `default`。

注意别拿逻辑运算符去替代 `if-else` 做复杂判断，代码会变得难读。而且它们的优先级比比较运算符低，写复合条件时记得加括号。

什么是 Python 面向对象的封装特性？

封装的本质是把对象的属性和操作这些属性的方法绑在一起，对外隐藏内部实现细节。你调用一个方法时，压根不关心它内部怎么实现的，只要知道这个接口能做什么就行。

比如定义一个类，用双下划线 `__` 开头的变量或方法，Python 会做名称重整（name mangling），让外部直接访问变得困难，这就实现了访问控制。但这不是绝对安全的，只是语法层面的限制。

- 1) 属性私有化：通过 `__attribute` 让属性不能被直接访问
- 2) 提供公共接口：用 `get_value()` 和 `set_value()` 这样的方法间接操作数据
- 3) 内部逻辑可变：外面用的人不需要改代码，因为底层实现可以随时调整

```
class BankAccount:
    def __init__(self):
        self.__balance = 0 # 私有属性

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance
```

像 Django 的模型字段、Flask 的请求上下文，都用了封装来屏蔽复杂性。外面拿到的是个简单接口，背后可能涉及数据库连接、线程局部存储这些脏活累活。

其实封装最大的好处是降低耦合。一个模块搞不定，拆成多个小部分，各自把自己封装好，别人用起来就省心。Java 里有 `private` 关键字，Python 更靠约定，但思路是一样的。

什么是 Python 面向对象的抽象特性？

面向对象的抽象特性，说白了就是把一类事物的共性抽出来，定义成一个“模板”，让子类去具体实现。Python 里没有原生的抽象类支持，但通过 `abc` 模块能强制实现这层约束。

- 1) 定义抽象类必须继承 `ABC`，并且用 `@abstractmethod` 装饰器标记方法。这样子类如果没重写这个方法，实例化时直接报错。
- 2) 抽象方法可以有实现，但一般不写具体逻辑，因为它的作用是“占位”，确保子类提供统一接口。

比如做支付系统，你可以定义一个抽象类 `Payment`，里面有个抽象方法 `pay()`。微信支付、支付宝支付这些子类就必须实现自己的 `pay()`，不然代码跑不起来。

```
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount):
        pass

class WeChatPay(Payment):
    def pay(self, amount):
        print(f"微信支付 {amount} 元")
```

这种设计在框架里很常见，像 Django 的视图类、Flask 的扩展机制，都是靠抽象来保证接口一致性。你不用关心每个子类怎么实现，只要知道它们都有 `pay` 这个能力就行。

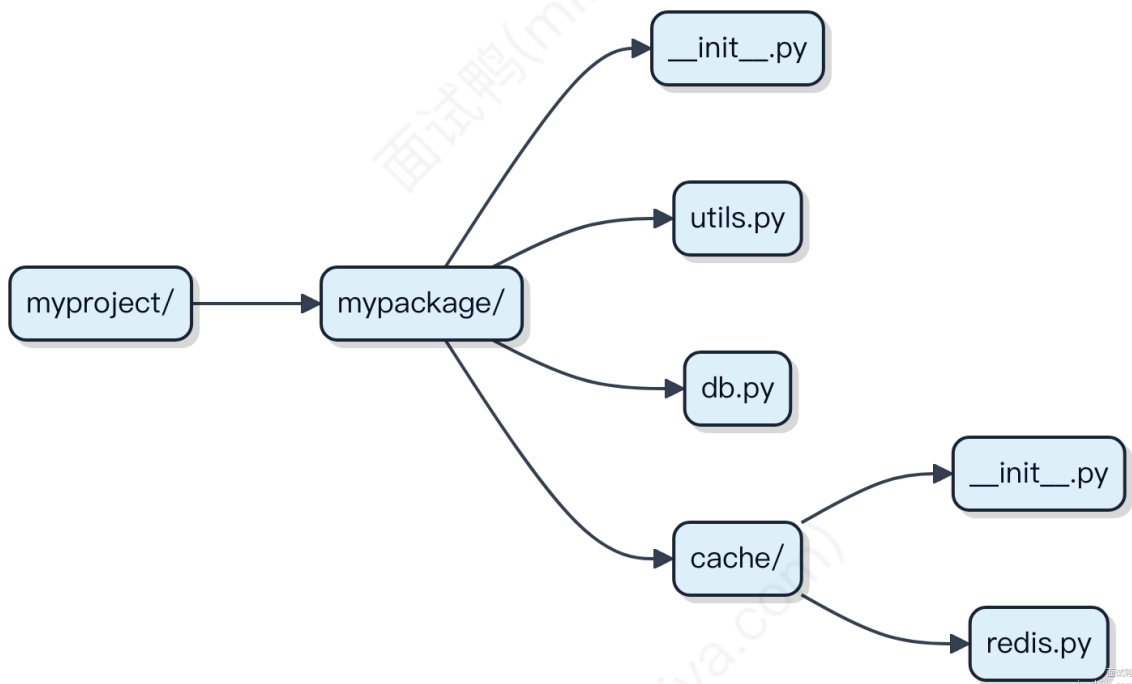
抽象不是为了“炫技”，而是为了**规范行为**。当你需要一组类对外表现一致、内部实现各异时，就该想到它。否则干脆用普通类，别给自己加戏。

什么是 Python 中的模块和包？

Python 里，一个 `.py` 文件就是一个模块，它能定义函数、类和变量，把相关代码组织在一起。比如写个 `utils.py`，里面放几个工具函数，别的文件要用时直接 `import utils` 就行。

多个模块可以进一步组织成包，用来管理更大规模的项目。包其实就是一个目录，但必须包含一个 `__init__.py` 文件（哪怕这个文件是空的），Python 才会把它识别为包。比如你有 `mypackage/` 目录，下面放了 `db.py` 和 `net.py`，再加个 `__init__.py`，就可以通过 `from mypackage import db` 这种方式导入。

包支持层级结构，也就是子包。像 `mypackage/cache/redis.py` 这样的路径，只要每一级目录都有 `__init__.py`，就能用 `import mypackage.cache.redis` 来引用。



实际项目中，像 Django 或 Flask 这些框架都大量使用包来组织代码。现在新项目虽然可以用隐式命名空间包（不用强制 `__init__.py`），但显式写出来更清晰，兼容性也好。

什么是 Python 的 OOPS（面向对象编程）？

面向对象编程在 Python 里其实是一种组织代码的方式，把数据和操作数据的方法绑在一起，构成“对象”。每个对象是某个类的实例，类就像图纸，定义了对象该有哪些属性和行为。

1) 类和实例

类用 `class` 定义，里面可以有属性和方法。实例是根据类创建出来的具体对象。

```
class Dog:
    def __init__(self, name):
        self.name = name

    def bark(self):
        print(f"{self.name} 叫了一声")
```

```
my_dog = Dog("旺财")
my_dog.bark()
```

2) 四大特性

封装是把内部细节藏起来，只暴露接口。比如用双下划线 `__private` 让属性难以直接访问。

继承让子类复用父类的代码，Python 支持多继承，但容易出问题，一般推荐用组合。

多态指的是不同类的对象可以对同一方法名做出不同响应。比如 `len()` 能作用于字符串、列表等不同类型。

抽象虽然 Python 没有原生支持，但可以通过 `abc` 模块定义抽象基类，强制子类实现某些方法。

3) 使用场景

适合业务模型清晰、状态和行为耦合强的场景，比如 Web 框架中的用户、订单等实体。小脚本或简单逻辑没必要硬套 OOP，函数式或过程式更直接。

什么是 Python 类中的 self?

Python 类里的 `self` 其实就是实例本身，你调用方法时 Python 会自动把实例传进去。

定义方法时写 `self` 是为了能访问实例的属性和其它方法。比如你有个类 `Person`，写方法就得通过 `self.name` 来读写这个实例的名字。

```
class Person:
    def __init__(self, name):
        self.name = name # 这里的 self 就是将来创建的 p1 或 p2

    def greet(self):
        return f"Hi, I'm {self.name}"

p1 = Person("Alice")
p1.greet() # 调用时, Python 自动把 p1 当作 self 传进 greet 方法
```

你不手动传 `self`，它是解释器帮你处理的绑定机制。换个名字语法上也可行，但所有人都用 `self`，别自找麻烦。

它和 Java 的 `this` 差不多意思，只不过 Python 要显式写在参数列表里，不能省。

Python 的类和对象有什么区别?

Python 里定义一个类，其实就是在描述一类事物的模板。比如你写个 `class Person:`，这不代表某个具体的人，而是定义了“人”这个类型该有的属性和行为。

当你用这个类去创建实例，比如 `p = Person()`，这时候 `p` 就是一个对象。它是根据类这个模板实实在在造出来的东西，有具体的内存空间，可以存储数据，也能调用方法。

- 1) 类是代码层面的定义，只在定义时解析一次，不占运行时的实例数据空间
- 2) 对象是类的运行时实例，每个对象都有自己的一套属性值，互不影响
- 3) 类可以看作对象的工厂，通过它能创建出多个对象

```
class Dog:
    def __init__(self, name):
        self.name = name
```

```
def bark(self):
    print(f"{self.name} is barking")

# 类 Dog 是定义
# 下面两个是对象，各有自己的 name 属性
d1 = Dog("旺财")
d2 = Dog("小黑")
```

所有对象都共享类中定义的方法，但每个对象的属性独立。你可以理解为：类是图纸，对象是按图纸造出来的机器，每台机器状态不同，但功能结构一致。

请解释 Python 线程池的工作原理？

Python 线程池的核心是预先创建一组线程，统一管理和复用它们来执行多个任务，避免频繁创建销毁线程带来的开销。操作系统创建和调度线程是有成本的，特别是在处理大量短任务时，这种开销会显著影响性能。

线程池里维护了一个任务队列和一组工作线程。所有提交的任务都会被放进这个队列，空闲的工作线程就从队列里取任务执行。你可以把它想象成一个快递中转站，快递员（线程）固定，包裹（任务）来了就放分拣区（队列），谁有空谁去处理。

Python 的 `concurrent.futures.ThreadPoolExecutor` 就是标准实现。比如你设置最大 5 个线程，哪怕提交 100 个任务，也最多只有 5 个线程在跑，剩下的任务排队等空闲线程。

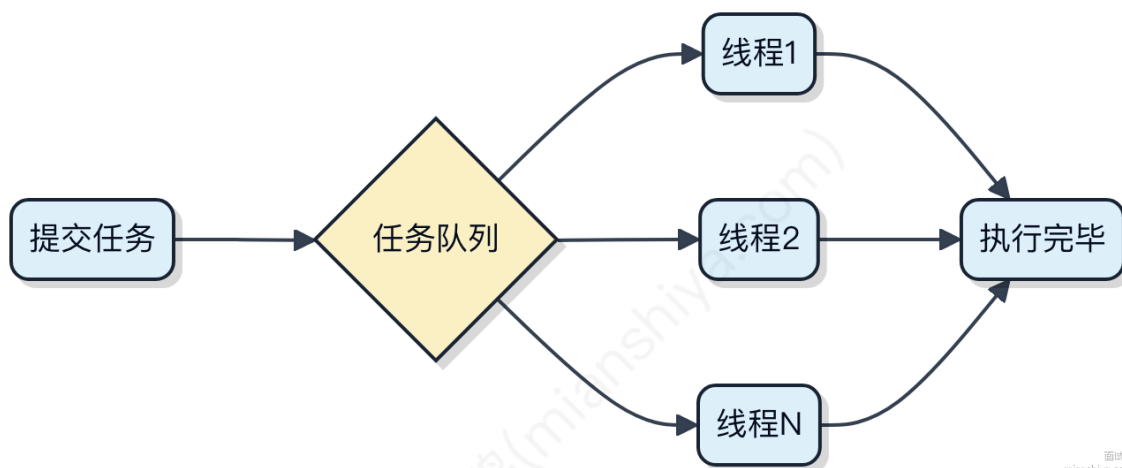
```
from concurrent.futures import ThreadPoolExecutor

def task(n):
    return n * n

with ThreadPoolExecutor(max_workers=5) as executor:
    results = list(executor.map(task, range(10)))
```

这里 `max_workers` 控制并发度。任务数超过线程数时，多出来的任务不会立刻执行，而是等待前面的任务完成。这种模式特别适合 I/O 密集型场景，比如用 `requests` 批量抓取网页，线程大部分时间在等网络响应，CPU 空闲，多线程能有效提升吞吐。

但要注意，由于 GIL 的存在，Python 线程池对 CPU 密集型任务几乎没帮助，这时候得用进程池。另外线程不安全操作还是要加锁，线程池本身不解决这个问题。



面试鸭
mianshiya.com

什么是 Python 面向对象的多态特性？

多态在 Python 里其实指的是同一个接口，不同实例能表现出不同的行为。它不靠继承强制约束，而是依赖“鸭子类型”——只要对象有对应的方法，就能调用，压根不关心它是不是某个父类的实例。

比如你定义一个函数 `make_sound(animal)`，只要传进来的对象有 `sound()` 方法，就能正常执行。Animal 是基类也好，Dog、Cat 自己实现也行，甚至完全无关的 Robot 能叫也 OK。Python 的多态是运行时决定的，灵活性高，但编译期没法检查方法是否存在。

- 1) 不需要显式继承同一父类也能实现多态
- 2) 方法存在即可用，没有接口或抽象类的硬性要求（虽然可以用 ABC 模块定义）
- 3) 出错往往发生在运行时，调用不存在的方法直接抛 `AttributeError`

```
def make_sound(animal):
    animal.sound() # 只要你有 sound 方法，我就敢调

class Dog:
    def sound(self):
        print("汪")

class Cat:
    def sound(self):
        print("喵")

make_sound(Dog()) # 汪
make_sound(Cat()) # 喵
```

这种设计让代码扩展性强，写框架时特别有用，像 Flask 或 Django 里的插件机制就利用了这一点。但反过来，过度依赖会导致可读性下降，得靠测试和文档来兜底。

Python 是否支持多重继承？

Python 确实支持多重继承，一个类可以同时从多个父类继承属性和方法。这跟 Java 完全不同，Java 只允许单继承，靠接口弥补。

写法上很简单，定义类的时候把多个父类放进括号里就行：

```
class A:
    def method(self):
        print("A method")

class B:
    def method(self):
        print("B method")

class C(A, B):
    pass

obj = C()
obj.method() # 输出 "A method"
```

这里调用 `method` 会按 **MRO (Method Resolution Order)** 查找，也就是方法解析顺序。Python 使用 C3 线性化算法来确定继承顺序，保证每个类只被调用一次，避免菱形继承的问题。

可以通过 `C.__mro__` 查看解析顺序：

```
print(C.__mro__)
# (<class '__main__.C'>, <class '__main__.A'>, <class '__main__.B'>, <class
'object'>)
```

实际用的时候要小心方法冲突。比如 A 和 B 都有同名方法，那排在前面的优先。如果父类都实现了 `__init__`，记得用 `super()` 协调调用，不然可能某个父类的初始化被跳过。

多重继承适合做 `mixin` 模式，像 Django 里的 `CreateView` 就混入了多个通用行为。但别滥用，否则继承链一复杂，维护起来很头疼。

为什么 Python 执行速度慢，如何改进？

Python 慢这事，得从它的设计哲学说起。它追求的是开发效率，不是运行效率，所以很多东西都做了妥协。

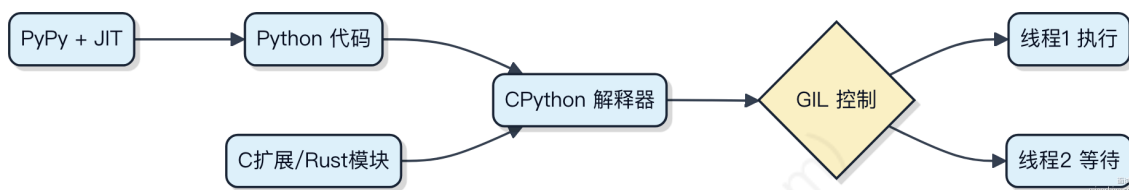
1) **动态类型**是性能杀手之一。变量类型在运行时才能确定，每次操作都要做类型检查和查找，不像 Java 或 C++ 在编译期就定死了。比如 `a + b`，解释器得先查 a 和 b 的类型，再找对应的加法实现，这个过程其实挺耗时间的。

2) **GIL (全局解释器锁)** 让 Python 的多线程没法真正并行。哪怕你有 16 核 CPU，CPython 执行字节码时也只允许一个线程跑，其他线程全在干等。CPU 密集型任务直接被锁住，压根不经过多核。

3) 解释执行本身也慢。Python 先把代码编译成字节码，再由解释器一条条执行，没有 JIT 编译优化，热点代码不会被编译成机器码加速。

改进办法看场景：

- **计算密集型**：用 Cython 把关键函数转成 C 扩展，或者上 Numba 直接装饰函数做 JIT。
- **并发处理**：绕开线程用 multiprocessing 走多进程，把 GIL 的问题直接避开。
- **Web 服务类**：换 PyPy，它自带 JIT，对纯 Python 逻辑提速明显，尤其长周期服务。
- **胶水角色**：让 Python 调用 C/C++ 或 Rust 写的扩展模块，像 pandas 底层就是 C 实现的。



本质上，Python 慢是因为它把脏活累活都揽在运行时。要提速，就得把这部分卸出去，交给更底层或更聪明的运行环境。

如何分析 Python 代码的执行性能？

分析 Python 代码性能，关键是从宏观到微观层层定位瓶颈。别一上来就看函数，先搞清楚是 I/O、CPU 还是内存拖慢了整体。

通常第一步用 `cProfile` 跑一遍程序，它能告诉你每个函数的调用次数、总耗时、每次平均耗时。比如：

```
import cProfile
cProfile.run('your_function()', 'output.stats')
```

生成的结果按累计时间排序，一眼看出最耗时的模块。这一步就能发现是不是某个第三方库在反复调用。

如果发现热点函数内部逻辑复杂，再上 `line_profiler`，精确到每一行。比如某次循环里频繁做字符串拼接，其实改成 `join` 就能从 200ms 降到 2ms。

I/O 密集场景得换思路。像处理大量文件或网络请求，用 `asyncio + aiohttp` 或者 `concurrent.futures` 做并发，可能直接从串行 10 秒压到并行 1 秒。但注意 GIL 的存在，纯 CPU 计算开多线程压根不加速，得用 `multiprocessing`。

内存问题用 `memory_profiler`，某对象是否一直不释放，有没有意外缓存累积。比如读大文件没用流式处理，一口气全 load 到内存，机器直接卡死。

最后，小范围验证优化效果，用 `timeit` 测几轮取平均：

```
import timeit
timeit.timeit(lambda: your_code(), number=1000)
```

性能分析其实是排除法，先定性再定量，工具链要会组合用。

Python 中如何读取大文件，例如内存只有 4G，如何读取一个大小为 8G 的文件

处理超大文件的核心是避免一次性加载到内存。Python 提供了多种流式读取方式，关键是控制每次读取的数据量。

用 `open()` 打开文件后，按固定大小分块读取最稳妥。比如设置每次读 1024 字节，循环处理直到文件结束。这样哪怕文件有几十 G，内存始终只占用几 KB 到几 MB。

```
with open('huge_file.txt', 'r') as f:
    while chunk := f.read(1024):
        process(chunk)  # 处理每一块数据
```

如果文件是按行组织的，比如日志文件，可以用迭代器逐行读取。 `for line in file` 实际上内部做了缓冲管理，也不会撑爆内存。

```
with open('huge_log.txt', 'r') as f:
    for line in f:
        process(line.strip())
```

生成器也能配合大文件使用。你可以写一个 `yield` 每一行或每一块的函数，在后续逻辑中像操作普通序列一样处理，但实际是惰性求值。

需要特别注意的是，不要用 `readlines()` 或 `list(f)` 这种方法，它们会试图把所有内容装进列表，8G 文件直接干掉 4G 内存。

文本编码也要提前确认好。遇到乱码导致读取出错时，可以加 `errors='ignore'` 或 `'replace'` 参数跳过异常字符。

什么是 Python 面向对象中的继承特点？

继承让子类能复用父类的属性和方法，同时可以扩展或修改行为。Python 支持多继承，一个类可以有多个父类，这和 Java 单继承不一样。

查找属性和方法时走的是 **MRO**（Method Resolution Order），也就是方法解析顺序。Python 用 C3 算法确定这个顺序，保证每个类只被访问一次，且父类在子类后面执行。可以用 `ClassName.__mro__` 查看顺序。

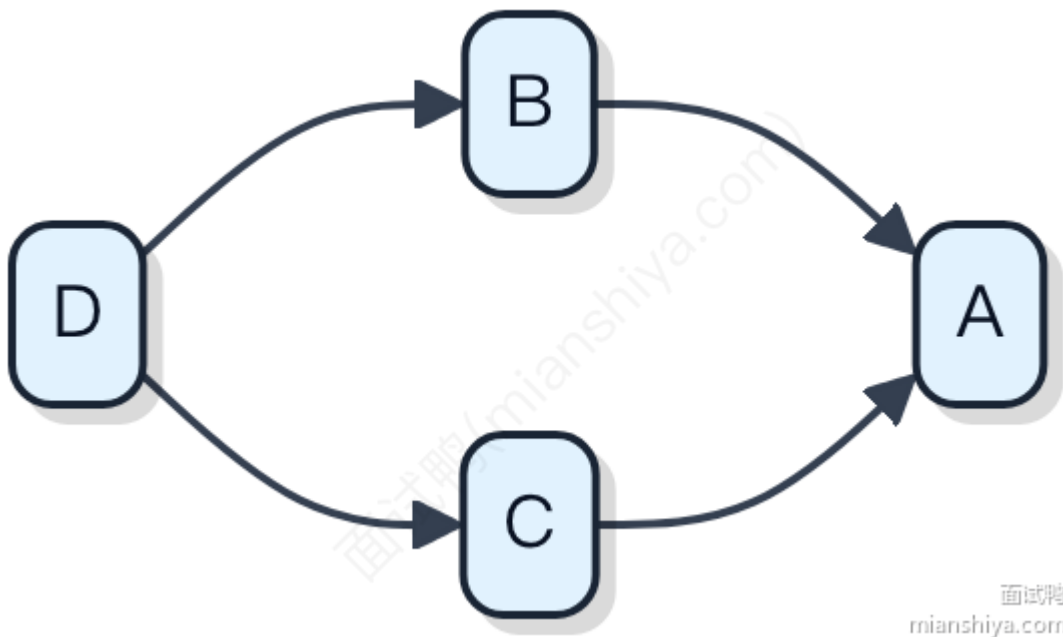
- 1) 子类可以重写父类的方法，实现多态
- 2) 通过 `super()` 调用父类方法，避免直接写父类名，增强维护性
- 3) 多继承时要注意方法冲突，合理设计类层次

```
class A:
    def greet(self):
        print("Hello from A")

class B(A):
    def greet(self):
        super().greet()
        print("Hello from B")
```

上面代码里 B 类调用父类 `greet`，再追加自己的逻辑，这是常见协作方式。

如果多个父类都有同名方法，按 MRO 顺序决定谁先被调用。设计时尽量避免菱形继承带来的歧义，但真用了 Python 也能正确处理。



图示为典型的菱形继承结构，D 继承 B 和 C，B 和 C 都继承 A。MRO 为 (D, B, C, A, object)，C3 算法确保 A 只出现一次。

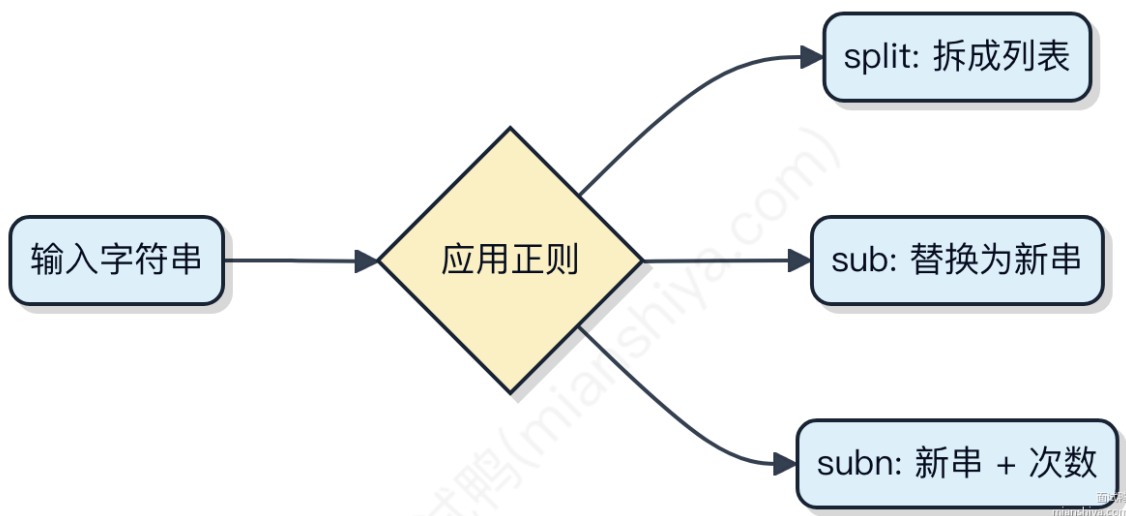
Python 的 re 模块中 split()、sub()、subn() 方法有什么作用？

`re.split()` 按正则表达式拆分字符串，返回列表。比如用 `\W+` 能把句子按非字母符号切开，像 `re.split(r'\W+', 'hello, world!')` 得到 `['hello', 'world', '']`。

- 1) `re.sub()` 替换匹配内容，返回新字符串。第三个参数是原字符串，还能指定替换次数。比如把所有数字替换成 *：`re.sub(r'\d', '*', 'a1b2c3')` 结果是 `'a*b*c*'`。
- 2) `re.subn()` 和 `sub()` 基本一样，但它返回的是一个元组，包含新字符串和替换次数。想知道改了几处就用它，比如 `re.subn(r'\d', '*', 'a1b2c3')` 返回 `('a*b*c*', 3)`。

这三个方法第一个参数都是正则模式，处理前会先编译。如果要重复使用同一个模式，建议提前用 `re.compile()` 编译好再调对应方法，性能更好。

实际写爬虫或日志清洗时特别常用。比如 Nginx 日志按空格和方括号分割字段，或者把敏感词打码替换，都是这类函数的典型场景。



Python 的 namedtuple 有什么作用？怎么使用？

namedtuple 是标准库 `collections` 里的一个工厂函数，用来定义一种轻量级的类，本质是元组的子类，但能通过字段名访问元素，比普通元组可读性强得多。

比如你要表示一个二维点，用普通元组得写 `point[0]`，`point[1]`，别人看不懂哪个是 x 哪个是 y。用 namedtuple 就清晰了：

```
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
print(p.x, p.y) # 输出: 3 4
```

它生成的类自带 `__repr__`、支持拆包、比较操作，而且创建开销极小，适合做数据记录载体。像解析 CSV 或 JSON 时，用它比字典省内存，又比元组易读。

注意它不可变，和 tuple 一样。如果需要修改字段，得用 `_replace()` 创建新实例：

```
p2 = p._replace(x=5)
```

也支持默认值（从 Python 3.7 开始）：

```
Person = namedtuple('Person', ['name', 'age'], defaults=[20])
```

常见于配置项、函数返回多个值但想命名的场景。像 Flask 或 Django 内部就拿它临时封装请求上下文数据。比起 class 定义，这是个轻量且安全的选择。

Python 中 read、readline、readlines 有哪些区别？

这三个方法都是用来读文件的，但行为差别挺大，用错了容易把内存搞崩或者读不到想要的内容。

`read()` 是一把梭，直接把整个文件从头到尾全读进内存，返回一个字符串。如果你处理的是小文件，比如几KB的配置，没啥问题。但要是读个1GB的日志，内存直接就炸了。一般不用它读大文件。

`readline()` 每次只读一行，返回字符串，读完返回空字符串。它是逐行加载，内存友好，适合处理大文件。比如你用 `for` 循环遍历文件对象，底层其实就是调 `readline()`，一行一行来，不会压垮内存。

`readlines()` 看着像复数版 `readline`，其实是把所有行一次性读进来，返回一个列表，每一行是列表里的一个元素。和 `read()` 一样，会把整个文件载入内存，只是结果按行切好了。除非你真需要随机访问某一行，不然没必要用它。

实际用的时候，大多数情况推荐这样写：

```
with open('log.txt') as f:
    for line in f: # 一行一行读，安全高效
        process(line)
```

这种方式底层是迭代器，不占内存，代码也干净。

[Python 中如何实现多线程？](#)

Python 的多线程靠 `threading` 模块来实现，但有个关键点必须清楚：**GIL**（全局解释器锁）的存在让同一时刻只有一个线程在执行 Python 字节码。

这意味着 CPU 密集型任务用多线程基本没用，压根不提升性能。你开了 10 个线程跑计算，还是只有一个能干活，其他都在等 GIL。

但 I/O 密集型场景就合适了。比如你用 `requests` 发 HTTP 请求，或读写文件，线程卡在等网络响应的时候会释放 GIL，其他线程就能进来处理自己的 I/O 任务。这种时候多线程能有效提升吞吐。

```
import threading
import requests

def fetch_url(url):
    resp = requests.get(url)
    print(f"{url}: {len(resp.content)}")

# 开两个线程并发请求
t1 = threading.Thread(target=fetch_url, args=("https://httpbin.org/delay/1",))
t2 = threading.Thread(target=fetch_url, args=("https://httpbin.org/delay/1",))

t1.start()
t2.start()
t1.join()
t2.join()
```

真正要做并行计算，得绕开 GIL。一般用 `multiprocessing` 模块，起多个进程，每个进程独立的解释器和内存空间，才能真正利用多核。

总结一下：I/O 多发任务用 `threading`，计算密集上 `multiprocessing`，别被“多线程”三个字带沟里去了。

[请介绍 Python 中多线程和多进程的应用场景，以及优缺点？](#)

Python 的多线程和多进程选择，关键看任务是受 **CPU 密集型** 还是 **I/O 密集型** 影响。

1) CPU 密集型任务比如科学计算、图像处理，这种要榨干 CPU 的场景，多线程在 Python 里基本没用。因为 **GIL (全局解释器锁)** 的存在，同一时刻只有一个线程能执行 Python 字节码，线程再多也压根不经过 CPU 并行。这时候就得上多进程，每个进程有独立的 Python 解释器和内存空间，真正利用多核。但代价是进程开销大，内存占用高，通信还得靠 `multiprocessing.Queue` 或共享内存。

2) I/O 密集型任务比如网络请求、文件读写、数据库操作，大部分时间在等响应。这种场景多线程就很合适，一个线程卡住 I/O，其他线程可以继续跑。像 `requests` 发 HTTP 请求，用 `concurrent.futures.ThreadPoolExecutor` 起个 10 来个线程，轻松扛住几十个并发请求，资源消耗远小于多进程。

代码上看区别也很直接：

```
# 线程池，适合 I/O 操作
with ThreadPoolExecutor(max_workers=5) as executor:
    executor.map(fetch_url, urls)

# 进程池，适合 CPU 计算
with ProcessPoolExecutor(max_workers=4) as executor:
    executor.map(compute_heavy_task, data)
```

简单说：**I/O 多线程，CPU 多进程**。这是 Python 下并发编程的基本原则。搞不定 GIL 就别在线程里做计算，不然白忙活。

什么是 Python 的 pickling 和 unpickling?

Python 的 pickling 是把 Python 对象转成字节流的过程，方便存文件或通过网络传输。反过来，unpickling 就是把字节流重新还原成原来的 Python 对象。

这个机制底层靠的是 `pickle` 模块实现，它能处理大多数内置类型和自定义类实例。比如你有个字典或者一个训练好的机器学习模型（像 `scikit-learn` 的模型），想保存下来以后用，直接 `pickle` 到文件就行。

1) 序列化 (Pickling) 示例：

```
import pickle

data = {'name': 'Alice', 'age': 30}
with open('data.pkl', 'wb') as f:
    pickle.dump(data, f)
```

2) 反序列化 (Unpickling) 示例：

```
with open('data.pkl', 'rb') as f:
    data = pickle.load(f)
```

要注意的是，**安全性**是个大问题。unpickling 数据相当于执行任意代码，如果来源不可信，可能被攻击。所以别对不信任的输入做 unpickle。

另外，`pickle` 是 Python 特有的，和其他语言不兼容。如果你需要跨语言交互，用 `JSON`、`Protocol Buffers` 更合适。

适合的场景包括缓存计算结果、保存模型状态、进程间通信（比如用 Redis 存 Python 对象）。但别拿它当长期数据存储方案，版本变更可能导致加载失败。

什么是 Python 的闭包？

函数能记住它被定义时所处的环境，哪怕这个环境已经消失了。

- 1) 当一个内部函数引用了外层函数的变量，这个内部函数就变成了闭包。外层函数返回后，内部函数依然可以访问那些变量，不会因为外层函数调用结束就被回收。
- 2) 闭包的关键在于“延迟绑定”和“变量捕获”。Python 的作用域规则决定了内部函数会保留对外部自由变量的引用。
- 3) 常见用途比如实现装饰器、回调函数、状态保持。像 Flask 路由装饰器 `@app.route` 就利用了闭包来保存路径配置。

```
def make_multiplier(n):
    def multiplier(x):
        return x * n
    return multiplier

double = make_multiplier(2)
print(double(5)) # 输出 10
```

上面这段代码里，`multiplier` 函数记住了 `n=2`，即使 `make_multiplier` 已经执行完了。这就是闭包在起作用。

闭包让函数带上了上下文，相当于一个轻量级的对象实例。但要注意循环中创建闭包可能引发的陷阱，比如 `lambda` 在列表推导式中容易绑定到同一个变量引用。

Python 中 `join()` 和 `split()` 函数有什么区别？

`split` 是把字符串按分隔符拆成列表，`join` 是把列表元素拼接成一个字符串，两个方法功能相反，属于字符串处理的常用操作。

比如 `'a,b,c'.split(',')` 返回 `['a', 'b', 'c']`，反过来 `','.join(['a', 'b', 'c'])` 得到 `'a,b,c'`。注意 `join` 必须保证列表里的元素都是字符串，否则会报错。

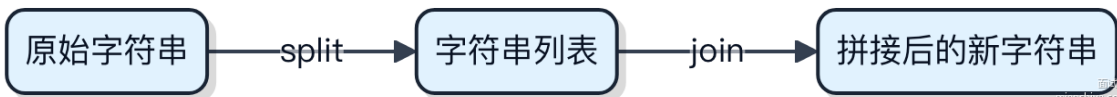
- 1) `split` 支持指定分隔符和最大分割次数

```
text = "apple,banana,grape"
parts = text.split(",", 1) # 只分割一次 -> ['apple', 'banana,grape']
```

- 2) `join` 的调用主体是分隔符字符串，参数是可迭代对象

```
'-'.join(['2024', '04', '01']) # 结果: '2024-04-01'
''.join(('x', 'y', 'z'))      # 结果: 'xyz'
```

常见坑是搞反了 `join` 的使用方式，写成 `lst.join(',')`，其实应该是 `','` 去调用 `join`。另外 `split` 在不传参数时默认按空白字符分割，包括空格、制表符、换行符，这点也容易被忽略。



Python 中 append、insert 和 extend 有什么区别？

append、insert 和 extend 都是 Python 列表操作的方法，但它们的用途和行为完全不同，搞混了容易出 bug。

append 是往列表末尾加一个元素，不管这个元素是啥类型，它都当做一个整体塞进去。比如 `lst.append([1, 2])`，结果列表里会多一个列表对象。

insert 是在指定位置插入一个元素，后面的元素自动往后挪。比如 `lst.insert(0, 'x')` 就是在开头插入，原来的元素都顺延，适合小规模调整顺序，频繁用在大列表上会影响性能。

extend 才是真正用来“扩展”列表的，它会把一个可迭代对象里的每个元素逐个拿出来，追加到原列表后面。比如 `lst.extend([1, 2, 3])` 相当于连续三次 append，但效率更高，代码也更干净。

举个例子：

```
lst = [1]
lst.append([2, 3])    # 结果: [1, [2, 3]]
lst.extend([4, 5])    # 结果: [1, [2, 3], 4, 5]
```

所以你要拼接两个列表，别用 append，用 extend 或 +=。append 和 insert 放一起说是因为它们都是加单个对象，而 extend 的核心是解包迭代。

什么是 Python 的字典，有哪些用法？

Python 的字典 (dict) 本质是一个哈希表，用键值对存储数据，查找平均时间复杂度是 $O(1)$ 。它不保证有序 (Python 3.7+ 因实现细节才保持插入顺序)，但能快速通过 key 拿 value。

创建字典很简单：

```
d = {'name': 'Bob', 'age': 25}
d = dict(name='Bob', age=25)
```

常用操作包括增删改查：

```
d['city'] = 'Beijing' # 增 / 改
print(d['name'])      # 查, key不存在会抛KeyError
print(d.get('job', 'N/A')) # 安全访问, 缺省返回None或指定值
del d['age']           # 删
```

遍历一般用：

```
for k, v in d.items():
    print(k, v)
```

注意点：字典的 key 必须是不可变类型，比如字符串、数字、元组；list 就不能当 key，因为 list 可变，哈希值没法固定。

适合场景：需要通过唯一标识快速查找数据时，比如缓存映射、配置管理、统计计数。像 Flask 的 `request.args` 就是字典结构处理 URL 参数。

避免在高并发场景直接用 dict，要加锁或用 `collections.defaultdict` 或 `Counter` 这类工具简化逻辑。

Python 中 remove、del 和 pop 有什么区别？

操作列表元素时，`remove`、`del` 和 `pop` 看似都能删东西，但用法和目的差得挺远。

`remove` 是按值删除，你告诉它“删掉哪个具体的值”，它去找第一个匹配的删了。比如 `list.remove(5)` 会删掉第一个等于 5 的元素。要是这值压根不存在，直接抛 `ValueError`。它不返回任何东西，就是默默删掉。

```
nums = [1, 2, 3, 2]
nums.remove(2) # 删第一个 2
print(nums)    # [1, 3, 2]
```

`del` 是个通用语句，能删列表里的单个元素、切片，甚至整个变量。它不关心值，只看位置。`del list[0]` 删第一个元素，`del list[1:3]` 删一段，`del list` 连变量都干掉。它也不返回被删内容。

```
nums = [10, 20, 30]
del nums[1] # 删除 20
print(nums) # [10, 30]
```

`pop` 是按索引删，默认删最后一个，还能把删掉的那个值返回回来。这就很适合用在“边取边删”的场景，比如当栈用的时候。`pop(0)` 虽然能删第一个，但性能差，因为要整体前移。

```
nums = [1, 2, 3]
last = nums.pop() # last = 3, nums = [1, 2]
first = nums.pop(0) # first = 1, nums = [2]
```

简单说：想删某个值就用 `remove`，想删位置且不关心返回值用 `del`，要删还得拿到删掉的值，就用 `pop`。

Python 中的 != 和 is not 运算符有什么区别？

`!=` 和 `is not` 看着都是做“不等于”判断，但它们比的压根不是一回事。

`!=` 比的是值。只要两个对象的内容相等，哪怕它们是不同的实例，结果就是 `False`。比如两个列表 `[1,2,3]` `!= [1,2,3]` 结果是 `False`，因为值一样。

而 `is not` 比的是身份，也就是内存地址。它只关心这两个变量是不是指向同一个对象。比如：

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a != b)    # False, 值相等
print(a is not b) # True, 不是同一个对象
```

所以你用 `is not` 的时候，其实是在问：“这俩是不是完全不同的东西？”而不是看内容一不一样。

一般情况下，比较数值、字符串内容就用 `!=`。只有当你明确想确认是不是同一个对象实例时才用 `is not`，比如判断是不是 `None`：

```
if x is not None:
    ...
```

这种写法是 Python 里的标准做法，因为 `None` 是单例，直接比身份高效又安全。

简单说：`!=` 看内容，`is not` 看是不是同一个东西。

Python 的 iterables 和 iterators 有什么区别？

Python 里可迭代对象和迭代器经常被混用，其实它们是两个不同层面的东西。

1) iterables 是能被 `for` 循环遍历的对象，比如 `list`、`tuple`、`str`、`dict`、`set`。它们的共同点是实现了 `__iter__()` 方法，调用后会返回一个迭代器。你平时写 `for x in [1,2,3]`，背后的机制就是先拿这个列表的迭代器。

2) iterators 才是真正实现“一次取一个”逻辑的对象。它必须有 `__iter__()` 和 `__next__()` 方法。`__next__()` 每次返回下一个值，到头了就抛 `StopIteration`。重点是，**迭代器本身也是可迭代的**，但它迭代自己时，`__iter__()` 直接返回自己，不会新建状态。

关键区别在于：可迭代对象每次 `for` 都能从头开始，因为每次都调 `__iter__()` 拿新迭代器；而迭代器一旦走到底，`__next__()` 就一直抛异常，相当于“耗尽”了。

```
nums = [1, 2, 3]
it = iter(nums) # 从 iterable 拿 iterator
print(next(it)) # 1
print(next(it)) # 2
```

生成器函数 `def gen(): yield 1` 返回的就是 **迭代器**，它不用手动写 `__next__`，语言帮你搞定了。这种设计把“数据来源”和“遍历逻辑”拆开，像 `itertools` 这种工具才能链式处理数据流。

Python 中的 Map 函数有什么作用？如何使用？

`map()` 是 Python 里的一个内置函数，用来对可迭代对象中的每个元素**批量执行**同一个函数操作。它不会修改原数据，而是返回一个新的迭代器，里面是处理后的结果。

用法很简单，第一个参数是函数名，后面跟一个或多个可迭代对象。比如你想把列表里每个数字转成字符串长度：

```
list(map(len, ['hello', 'world', 'python'])) # 输出 [5, 5, 6]
```

如果要对多个序列同时操作，比如把两个列表对应位置的数相加：

```
list(map(lambda x, y: x + y, [1, 2, 3], [4, 5, 6])) # 输出 [5, 7, 9]
```

注意，`map()` 返回的是 `map` 对象，不是列表。你要么用 `list()` 转一下，要么在循环里直接用。这在处理大数组时挺省内存的，因为它是惰性计算。

不过现在很多场景下大家更喜欢用列表推导式，代码看起来更直观。比如 `[x * 2 for x in data]` 比 `map(lambda x: x * 2, data)` 更易读。

但遇到复杂函数或者多个输入源时，`map()` 依然很实用。像数据清洗阶段，统一处理一批字段就很常见，比如用 `map(int, string_list)` 快速转类型。

别把它和 `filter()` 或 `reduce()` 搞混了，各干各的活。`map` 只负责“映射”，输入输出数量一定是一一对应的。

Python 中的 Filter 函数有什么作用？怎么使用？

`filter` 函数用来从可迭代对象里筛选出符合条件的元素，生成一个新的迭代器。它不改变原数据，而是把通过条件的元素挑出来。

用法是 `filter(函数, 可迭代对象)`。那个函数得返回布尔值，`True` 就留下，`False` 就过滤掉。如果第一个参数传了 `None`，那就直接看元素本身是不是真值，比如过滤掉 `False`、`0`、空字符串这类。

比如你要从列表里找出所有偶数：

```
nums = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, nums))
# 结果是 [2, 4, 6]
```

再比如去掉空字符串：

```
texts = ['hello', '', 'world', ' ', '']
filtered = list(filter(None, texts))
# 注意：' ' 不是空，有空格，所以结果会保留它
# 真正去掉空串还得用 lambda s: s.strip() if isinstance(s, str) else s
```

一般配合 `lambda` 用得更多，写匿名函数方便。但复杂逻辑还是单独定义函数更清晰。

要注意返回的是 `filter` 对象，不是列表，得手动转一下。这在处理大数据组时挺省内存的，支持惰性求值。

跟列表推导式比，`[x for x in data if cond]` 更直观，性能也通常更好，所以很多人宁愿用后者。但 `filter` 在函数已存在时代码更简洁，比如 `filter(str.isdigit, strings)` 这种一眼就懂。

Python 中 reduce 函数有什么作用？怎么使用？

`reduce` 的作用是把一个可迭代对象从左到右逐步“归约”成一个单一结果，每次拿前一次的计算结果和下一个元素继续运算。

它在 `functools` 模块里，调用方式是 `reduce(function, iterable[, initializer])`。`function` 是一个接收两个参数的函数，比如加法、乘法或者自定义逻辑。`iterable` 就是要处理的数据，比如列表。`initializer` 是可选的初始值。

举个例子，你想算 `[1, 2, 3, 4]` 的累加和：

```
from functools import reduce
```

```
result = reduce(lambda x, y: x + y, [1, 2, 3, 4])
# 计算过程: ((1 + 2) + 3) + 4 = 10
```

如果加上初始值，比如从 10 开始：

```
result = reduce(lambda x, y: x + y, [1, 2, 3, 4], 10)
# ((10 + 1) + 2) + 3) + 4 = 20
```

常见用途包括累积计算、链式判断、嵌套字典取值（配合 lambda 和 get），但别滥用。能用 for 循环说清楚的地方，就不用 reduce，否则代码容易变绕。

注意，Python 里很多场景其实有更直观的替代方案。比如求和直接用 `sum()`，连乘可以用 `math.prod()`（Python 3.8+），或者显式的循环反而更易读。

- 1) 没有初始值时，第一次会拿前两个元素做运算
- 2) 序列为空且没给初始值，会抛异常
- 3) 数据量大时注意函数调用开销，性能不如原生 C 实现的聚合操作

Python 在什么情况下会出现 `KeyError`、`TypeError`、`ValueError`？

`KeyError`、`TypeError`、`ValueError` 是 Python 里最常见的三种异常，基本写过几天代码就会遇到。

KeyError 出现在你试图从字典里拿一个不存在的键。比如 `d = {'a': 1}; print(d['b'])`，直接报 `KeyError`。处理办法一般是用 `.get()` 方法提供默认值，或者先用 `in` 判断键是否存在。

```
d.get('b', 0) # 安全访问，没找到返回 0
```

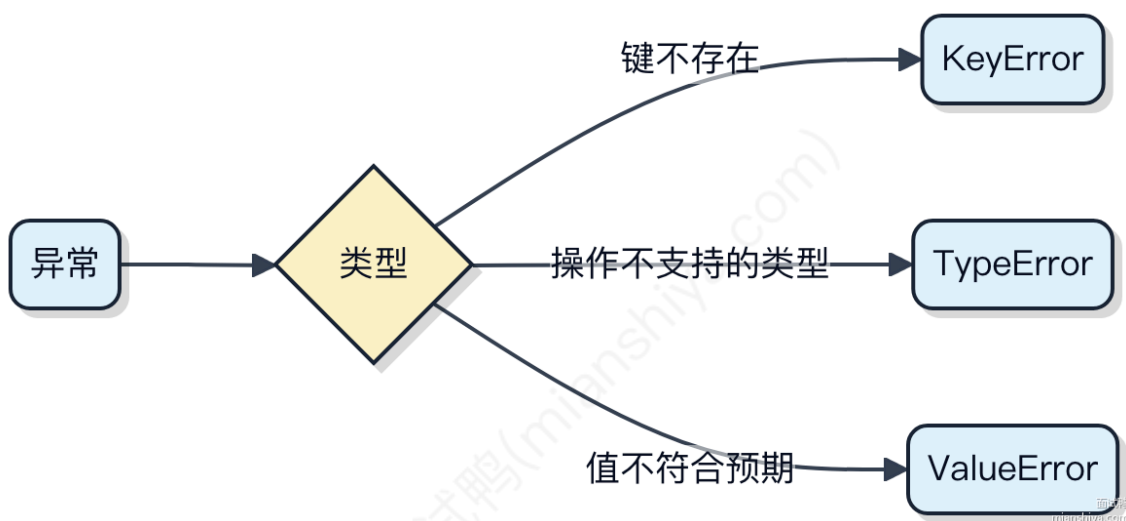
TypeError 是类型搞错了。典型场景是把字符串和数字相加 `1 + '2'`，或者把不可哈希的对象当字典 key 用，比如列表 `d[[1,2]] = 3`。还有一种是函数调用时参数类型不对，比如 `len(123)`。

ValueError 是类型对了，但值不合法。比如你调 `int('abc')`，字符串看着不像数字，转不了；或者 `json.loads('{key}')` 解析坏格式 JSON。还有像 `math.sqrt(-1)` 这种数学上不成立的操作，默认也会抛 `ValueError`。

简单记：

- 1) 找键找不到 → `KeyError`
- 2) 类型用错 → `TypeError`
- 3) 值不合规 → `ValueError`

这三类异常通常都能通过前置判断或 `try-except` 捕获来兜底，尤其在处理用户输入、配置解析、API 数据时特别常见。



什么是 Python 的生成器？

生成器是 Python 里一种特殊的函数，它能一边计算一边返回结果，而不是把所有值全算出来再给你。你调用它的时候不会立刻执行，而是返回一个**生成器对象**，你可以通过 `next()` 或者 `for` 循环来逐步取值。

它最大的特点是用 `yield` 关键字暂停函数状态。每次遇到 `yield`，函数就把当前的值交出去，自己“冻结”在那。下次再取值，就上次停的地方继续执行。

比如写个斐波那契数列的生成器：

```
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

g = fib()
print(next(g)) # 0
print(next(g)) # 1
print(next(g)) # 1
```

相比直接返回列表，生成器省内存多了。假设你要处理几百万个数据，用列表可能吃掉几百 MB，而生成器几乎不占内存，因为它只保存当前状态。

适合场景：读大文件、无限序列、数据流水线处理。像你在用 `requests` 流式下载大文件时，底层 `response.iter_content` 就是靠生成器实现的。

反过来，如果你需要重复遍历、或者随机访问元素，那就别用生成器，因为它只能往前走，不能回头。

Python 中的列表和元组有什么区别？

列表和元组最直观的区别在于**可变性**。列表是可变的，你随时可以增删改其中的元素；元组一旦创建，就不能再修改，这决定了它们底层实现和使用场景的不同。

因为元组长度固定且不可变，Python 在存储上更紧凑，访问速度也略快一些。这种不可变性还让元组可以作为字典的键，或者放进集合里去重，而列表不行。

使用场景上，如果数据是同质的、需要频繁增删的，比如一堆用户订单，用 list 更合适。如果是表示一个结构化的记录，比如坐标 (x, y)、数据库的一行记录，用 tuple 更语义清晰，也能防止意外修改。

代码上看区别：

```
# 列表：支持修改
orders = ["apple", "banana"]
orders.append("cherry") # 可以添加

# 元组：不能修改
point = (3, 5)
# point[0] = 4 # 这会报错！
```

性能方面，小量数据差别不大，但在大量短小序列场景下，比如函数返回多个值，用元组是 Python 的惯用法：

```
def get_name_age():
    return "Alice", 25 # 自动打包成元组

name, age = get_name_age() # 解包方便
```

- 1) 可变性决定是否能原地修改
- 2) 可变性进一步影响能否哈希
- 3) 语义用途不同：list 做容器，tuple 做记录

总的来说，该用哪个，关键看你要表达的是“一组东西”还是“一个整体”。

什么是 Python 的 Lambda 函数，有哪些应用场景？

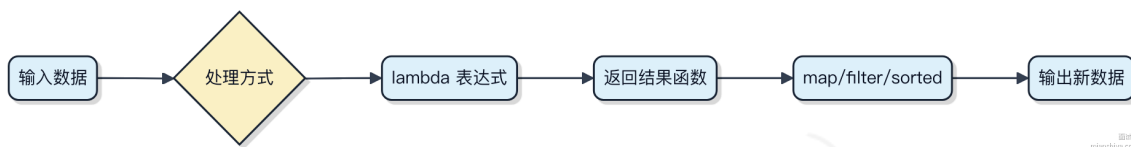
Lambda 函数是 Python 中的匿名函数，用一行表达式就能定义简单逻辑，不用写完整的 def 函数。它返回一个函数对象，常用于需要函数作为参数的场景。

语法就是 `lambda 参数: 返回值表达式`，比如 `lambda x: x * 2` 就是接收 x 返回它的两倍。这种写法简洁，适合那种只用一两次、逻辑又特别简单的函数。

- 1) 常配合高阶函数使用。比如 `map()` 对列表每个元素做处理，`list(map(lambda x: x.upper(), ['a', 'b']))` 把所有字符串转大写。
- 2) `filter()` 按条件过滤，`list(filter(lambda x: x > 0, [-1, 0, 1]))` 留下正数。
- 3) `sorted()` 自定义排序规则，`sorted(names, key=lambda name: len(name))` 按名字长度排。

不过别滥用。如果逻辑超过一两步，或者要写 if-else 嵌套，那就该用普通函数了。代码可读性更重要，`def` 能加文档字符串、类型提示，调试也方便。

另外 lambda 只能包含表达式，不能有 while、for、return 这些语句。想在 lambda 里赋值也不行，除非用海象运算符 (Python 3.8+)，但那样代码会很难懂，不推荐。



总的来说，lambda 的价值在于**轻量**和**即用即弃**。写得克制，能让代码更紧凑；写得过头，就会变成别人看不懂的“聪明代码”。

Python 的迭代器和生成器有什么区别？

迭代器和生成器都用来做数据的遍历，但实现方式和使用成本差挺多。

1) 迭代器是通过实现 `__iter__` 和 `__next__` 方法的对象。每次调用 `next()` 返回一个值，到头了就抛 `StopIteration`。你写类的时候手动控制状态，比如遍历一个列表或者读文件一行行处理，逻辑清晰但代码多。

```
class Counter:
    def __init__(self, low, high):
        self.current = low
        self.high = high

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.high:
            raise StopIteration
        self.current += 1
        return self.current - 1
```

2) 生成器是 Python 的语法糖，函数里用了 `yield` 就会变成生成器函数，调用后返回一个**生成器对象**，它自动满足迭代器协议。你不用管状态怎么存，`yield` 暂停函数，下次 `next()` 再继续。代码简洁很多，大多数场景优先用这个。

```
def counter(low, high):
    while low <= high:
        yield low
        low += 1
```

关键区别在于，生成器是迭代器的子集，所有生成器都是迭代器，反过来不成立。生成器帮你把脏活累活——状态维护、异常抛出——全包了，写起来更轻量。比如处理大文件或无限序列，用生成器能省内存，因为它是按需计算，压根不一次性加载所有数据。

一般情况下，能用生成器就不用手写迭代器，除非你需要复杂的内部状态管理。

Python 正则表达式中 `match` 和 `search` 有什么区别？

`match` 和 `search` 都是 Python `re` 模块用来匹配字符串的方法，但它们的行为有关键差异。

match 只从字符串的**开头**尝试匹配。如果字符串起始位置不满足模式，哪怕后面有完全匹配的部分，它也返回 None。这就像锚定在了最前面，压根不往后看。

search 则会扫描整个字符串，一旦发现某个位置能匹配上就立即返回结果。它不关心是不是开头，只要有地方对得上就行。

举个例子：

```
import re

text = "abc123def"
pattern = r"\d+"

re.match(pattern, text)    # 返回 None，因为开头不是数字
re.search(pattern, text)   # 匹配成功，找到 '123'
```

所以用哪个取决于你的需求。如果你明确要验证字符串是否以某种模式开始，比如判断日志行是否以时间戳开头，用 match 更合适。而当你需要在文本中找出第一个符合规则的片段，比如从一段话里提取手机号，就得靠 search。

常见误区是以为 match 会全局查找，其实它只看起点。反过来，search 虽然灵活，但如果想确保整体结构合规，就不能替代 match。

Python 中深拷贝和浅拷贝有什么区别？

Python 里拷贝对象分两种情况，浅拷贝和深拷贝，关键看嵌套对象的处理。

1) 浅拷贝只复制最外层对象，里面的子对象还是引用原来的。比如列表里有个字典，拷贝后新旧列表共用这个字典。改里面的字典，两个列表都会看到变化。

```
import copy
a = [1, {'x': 2}]
b = copy.copy(a) # 浅拷贝
b[1]['x'] = 99
print(a[1]['x']) # 输出 99，a 被影响了
```

2) 深拷贝是彻底复制，从外到内所有层级都新建一份。原对象和副本完全独立，互不干扰。适合数据结构复杂、嵌套多层的场景。

```
c = copy.deepcopy(a)
c[1]['x'] = 88
print(a[1]['x']) # 还是 99，a 不受影响
```

一般情况下，不可变类型像数字、字符串无所谓深浅拷贝。问题常出在 list、dict、自定义对象这种可变又可能嵌套的结构上。你要是搞不定到底用哪个，保险起见直接用 **deepcopy**，但代价是速度慢点，内存多耗些。

小项目数据简单，浅拷贝够用。Django 处理表单数据或 Flask 里传 context 的时候，如果涉及嵌套结构修改，就得小心是不是得上深拷贝。

Python 的 ****init**** 和 ****new**** 方法有什么区别？

`__new__` 是真正创建实例的方法，而 `__init__` 只是负责初始化已创建的实例。

1) `__new__` 是一个类方法，它在实例创建之前被调用，接收的第一个参数是类本身，并返回一个该类的实例。如果你想要控制对象的创建过程，比如实现单例模式或不可变类型，就得重写它。它压根不关心初始化逻辑，只管“造出”一个空对象。

2) `__init__` 是在 `__new__` 返回实例之后自动调用的，它的作用是给这个已经存在的实例“填数据”。你常见的属性赋值 `self.name = name` 都放在这里。它没有返回值，只是“装修”对象。

典型场景是自定义元类或者需要缓存实例时操作 `__new__`，比如 `type.__new__` 在构建类的时候起作用。大多数情况下我们只用 `__init__`，因为不需要干预对象创建过程。



像 `int`、`str` 这些不可变类型，如果要定制创建行为，必须通过 `__new__` 来做，因为 `__init__` 修改不了已经生成的值。而普通类里改 `__new__` 多半是为了拦截或复用实例，比如全局唯一配置对象。

Python 函数参数 `*args` 和 `**kwargs` 有什么区别？怎么使用？

`*args` 和 `**kwargs` 是 Python 函数参数的两种打包方式，用来处理不确定数量的参数。

1) `*args` 把位置参数收集成一个元组。你调用函数时传多少个位置参数都行，它们会自动变成一个 tuple。比如：

```
def func(*args):
    print(args) # 输出 (1, 2, 3)

func(1, 2, 3)
```

这个在封装通用逻辑时特别有用，比如写装饰器或者代理方法。

2) `**kwargs` 则收集关键字参数，构造成一个字典。只能接收 `key=value` 这种形式的参数。

```
def func(**kwargs):
    print(kwargs) # 输出 {'a': 1, 'b': 2}

func(a=1, b=2)
```

它常用于配置类接口，像 Flask 的路由参数、Django 表单字段定义都在用这种模式。

两者可以同时出现在函数签名里，顺序必须是：普通参数 -> `*args` -> `**kwargs`。因为解析时先匹配明确的，再按规则分流到 `args` 和 `kwargs`。

实际场景中，很多框架用它们做透传。比如你写个中间层要调下游函数，但不想把所有参数列全，就可以用 `def wrapper(*args, **kwargs): return real_func(*args, **kwargs)`。这样不管底层怎么变，上层都能透传过去，压根不经过额外处理。

注意别滥用，除非真需要灵活性。否则会让函数签名不清晰，调用者搞不懂该传啥。

Python 2 和 Python 3 有什么区别？

Python 2 和 Python 3 的区别不是小升级，而是一次**不兼容的大换血**。很多语法和行为都变了，迁移项目时得格外小心。

- 1) 最明显的差异在 `print`。Python 2 把它当语句，写 `print "hello"` 没问题；Python 3 改成了函数，必须写 `print("hello")`，不然直接报错。
- 2) 字符串和编码处理彻底重构。Python 2 的字符串类型混乱，`str` 是字节，`unicode` 才是文本，经常出乱码。Python 3 默认用 **Unicode**，`str` 就是文本，`bytes` 才是字节，编码更清晰。
- 3) 除法行为也变了。Python 2 中 `5 / 2` 得 `2`（整数除），要浮点得写 `5.0 / 2` 或导入 `division` 模块。Python 3 里 `/` 直接返回浮点，`//` 才是地板除。
- 4) `range()` 在 Python 2 返回列表，内存消耗大；Python 3 返回的是可迭代对象，类似生成器，省内存。同理，`map()`、`filter()` 也变成惰性求值。
- 5) 异常捕获语法微调。Python 2 支持 `except Exception, e`，Python 3 必须用 `except Exception as e`，前者会被抛语法错误。
- 6) 一些内置库被重命名或重组。比如 `urllib2` 被合并进 `urllib`，`ConfigParser` 改名 `configparser`（全小写）。

代码示例：

```
# Python 3
with open('中文.txt', 'w', encoding='utf-8') as f:
    f.write('你好')
```

这种默认支持 UTF-8 的写法，在 Python 2 里搞不定。现在主流框架如 Django、Flask 都只支持 Python 3，官方也在 2020 年停止维护 Python 2，新项目压根不用考虑老版本。

什么是“猴子补丁”（monkey patching）？

“猴子补丁”指的是在程序运行时动态修改类或模块的行为，不用改源码也能让老代码执行新逻辑。这招在 Python 里特别常见，Java 这种静态语言基本没这说法，得靠字节码增强比如 ASM、Javassist 或 Spring 的 AOP 来实现类似效果。

一般用在框架层面做透明增强，比如你想给某个第三方库的函数加个日志埋点，但又不想 fork 项目自己维护，就可以应用启动时“打补丁”，把原方法替掉或者包装一层。

举个 Python 的例子：

```
class Service:
    def fetch_data(self):
        return "old logic"

def new_fetch_data(self):
    return "new logic"

# 运行时替换
Service.fetch_data = new_fetch_data
```


看起来很方便，但也容易搞出幺蛾子。多个地方都去改同一个方法，冲突了很难查；调试时发现实际执行的不是源码里的逻辑，直接懵圈。所以一般来说，业务代码别乱用，框架级工具可以谨慎考虑。

- 1) 适合做无侵入监控、埋点、测试 mock
- 2) 生产环境慎用，后期维护成本高
- 3) Python、Ruby 这类动态语言支持好，JavaScript 原型链也能实现类似操作



本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)